```
# UZ Номерок: сервер игры (FastAPI + SQLite). Запускается через start.py вместе с
# Запросы к базе идут в потоках (сайт не «замирает»), соединения с базой постоянн
# проверка подписки кэшируется, база сама чистится и раз в сутки копируется в пап
import asyncio, gzip, hashlib, hmac, json, logging, math, os, random, shutil, sql
import urllib.error, urllib.parse, urllib.request
from collections import deque
from concurrent.futures import ThreadPoolExecutor
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import Response
from fastapi.staticfiles import StaticFiles

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(name)
logging.getLogger("httpx").setLevel(logging.WARNING)
log = logging.getLogger("uzn")

BOT_TOKEN = os.environ.get("BOT_TOKEN", "PASTE_BOT_TOKEN_FROM_BOTFATHER")
DB_PATH = os.environ.get("DB_PATH", "game.db")
TG_API = os.environ.get("TG_API", "https://api.telegram.org").rstrip("/")
# Каналы для призов за подписку: id (как в игре, CHS в html) -> @канал. Бот долже
CHANNELS = {"c1": os.environ.get("CHANNEL1", "@your_channel"), "c2": os.environ.g
REQUIRED_CHANNEL = os.environ.get("REQUIRED_CHANNEL", "@your_channel")
REQUIRED_URL = os.environ.get("REQUIRED_URL", "https://t.me/your_channel")   # cc
BOT_USERNAME = os.environ.get("BOT_USERNAME", "your_bot")                    # ло
REF_MILESTONES = [(1, 3000000), (3, 10000000), (5, 25000000), (10, 100000000)]  #
REF_LUCK = (20, 1440)   # 20 друзей -> удача ×2 на 24 часа (минуты)
HERE = os.path.dirname(os.path.abspath(__file__))
BK_DIR = os.environ.get("BACKUP_DIR") or os.path.join(os.path.dirname(os.path.abs
BK_KEEP = int(os.environ.get("BACKUP_KEEP", "5"))
ADMIN_IDS = [int(x) for x in os.environ.get("ADMINS", "").replace(";", ",").repla
APP_URL = os.environ.get("APP_URL", "")   # адрес игры (для кнопки «Открыть игру»
MAX_BODY = 3_000_000   # больше 3 МБ от игрока не принимаем

app = FastAPI(docs_url=None, redoc_url=None, openapi_url=None)

# ---------------------------------------------------------------- база данных
SCHEMA = """
CREATE TABLE IF NOT EXISTS users(id INTEGER PRIMARY KEY,name TEXT,first_seen INT,
CREATE TABLE IF NOT EXISTS gifts(id INTEGER PRIMARY KEY AUTOINCREMENT,uid INT,dat
CREATE TABLE IF NOT EXISTS promos(code TEXT PRIMARY KEY,reward TEXT,max_uses INT,
CREATE TABLE IF NOT EXISTS redeemed(code TEXT,uid INT,PRIMARY KEY(code,uid));
CREATE TABLE IF NOT EXISTS offers(id INTEGER PRIMARY KEY AUTOINCREMENT,frm INT,to
CREATE TABLE IF NOT EXISTS refpending(uid INTEGER PRIMARY KEY,ref INT);
CREATE TABLE IF NOT EXISTS referrals(uid INTEGER PRIMARY KEY,ref INT);
CREATE TABLE IF NOT EXISTS events(name TEXT,luck REAL,sell REAL,xp REAL,until INT
```

```
CREATE TABLE IF NOT EXISTS meta(k TEXT PRIMARY KEY,v TEXT);
CREATE TABLE IF NOT EXISTS bp(uid INTEGER,sk TEXT,tier INTEGER,PRIMARY KEY(uid,sk
CREATE TABLE IF NOT EXISTS deal_used(uid INTEGER,day INTEGER,PRIMARY KEY(uid,day)
CREATE TABLE IF NOT EXISTS uniq(key TEXT PRIMARY KEY,owner INT,ts INT);
CREATE TABLE IF NOT EXISTS auctions(id INTEGER PRIMARY KEY AUTOINCREMENT,seller I
CREATE TABLE IF NOT EXISTS shop(id INTEGER PRIMARY KEY AUTOINCREMENT,seller INT,i
CREATE TABLE IF NOT EXISTS car_tune(uid INT,cid TEXT,s INT DEFAULT 0,su INT DEFAU
CREATE INDEX IF NOT EXISTS ix_ctcid ON car_tune(cid);
CREATE TABLE IF NOT EXISTS cosm(uid INT,k TEXT,ts INT,PRIMARY KEY(uid,k));
CREATE TABLE IF NOT EXISTS rstat(uid INT,sk TEXT,played INT DEFAULT 0,wins INT DE
CREATE TABLE IF NOT EXISTS races(id INTEGER PRIMARY KEY AUTOINCREMENT,a INT,acar
CREATE TABLE IF NOT EXISTS upg_pend(id INTEGER PRIMARY KEY AUTOINCREMENT,uid INT,
CREATE INDEX IF NOT EXISTS ix_upgp ON upg_pend(uid);
CREATE TABLE IF NOT EXISTS duels(id INTEGER PRIMARY KEY AUTOINCREMENT,a INT,aplat
CREATE TABLE IF NOT EXISTS claims(uid INT,id TEXT,PRIMARY KEY(uid,id));
CREATE TABLE IF NOT EXISTS payments(charge TEXT PRIMARY KEY,uid INT,item TEXT,sta
CREATE INDEX IF NOT EXISTS ix_gifts_uid ON gifts(uid);
CREATE INDEX IF NOT EXISTS ix_offers_to ON offers(to_id,status);
CREATE INDEX IF NOT EXISTS ix_offers_frm ON offers(frm,status);
CREATE INDEX IF NOT EXISTS ix_ref_ref ON referrals(ref);
CREATE TABLE IF NOT EXISTS uniq_own(key TEXT,uid INT,ts INT,PRIMARY KEY(key,uid))
CREATE TRIGGER IF NOT EXISTS tr_uniq_ins AFTER INSERT ON uniq WHEN NEW.owner>0 BE
CREATE TRIGGER IF NOT EXISTS tr_uniq_upd AFTER UPDATE OF owner ON uniq WHEN NEW.o
INSERT OR IGNORE INTO uniq_own(key,uid,ts) SELECT key,owner,ts FROM uniq WHERE ow
CREATE INDEX IF NOT EXISTS ix_auc_open ON auctions(status,end);
CREATE INDEX IF NOT EXISTS ix_auc_seller ON auctions(seller,status);
CREATE INDEX IF NOT EXISTS ix_shop_open ON shop(status,id);
CREATE INDEX IF NOT EXISTS ix_shop_seller ON shop(seller,status);
CREATE INDEX IF NOT EXISTS ix_duel_open ON duels(status,id);
CREATE INDEX IF NOT EXISTS ix_duel_a ON duels(a,status);
CREATE INDEX IF NOT EXISTS ix_duel_b ON duels(b,status);
CREATE INDEX IF NOT EXISTS ix_uniq_owner ON uniq(owner);
CREATE INDEX IF NOT EXISTS ix_users_seen ON users(last_seen);
"""

def init_db():   # один раз при запуске: таблицы, индексы, режим WAL
    d = os.path.dirname(os.path.abspath(DB_PATH))
    if d: os.makedirs(d, exist_ok=True)
    c = sqlite3.connect(DB_PATH, timeout=30)
    try:
        c.execute("PRAGMA journal_mode=WAL"); c.executescript(SCHEMA)
        try: c.execute("ALTER TABLE upg_pend ADD COLUMN cnt INT DEFAULT 0")
        except Exception: pass
        for col_ in ("ae", "ni"):
            try: c.execute("ALTER TABLE car_tune ADD COLUMN %s INT DEFAULT 0" % c
            except Exception: pass
        try: c.execute("ALTER TABLE users ADD COLUMN bal INT DEFAULT 0")
```

```
        except sqlite3.OperationalError: pass
        try: c.execute("ALTER TABLE users ADD COLUMN stars INT DEFAULT 0")   # зв
        except sqlite3.OperationalError: pass
        c.execute("CREATE TABLE IF NOT EXISTS wlog(id INTEGER PRIMARY KEY AUTOINC
        c.execute("CREATE TABLE IF NOT EXISTS star_auc(id INTEGER PRIMARY KEY AUT
        c.execute("CREATE INDEX IF NOT EXISTS ix_sa_status ON star_auc(status,end
        c.execute("CREATE TABLE IF NOT EXISTS wallets(uid INTEGER PRIMARY KEY,add
        for q in ("ALTER TABLE users ADD COLUMN mute INT DEFAULT 0", "ALTER TABLE
            try: c.execute(q)
            except sqlite3.OperationalError: pass
        try: c.execute("ALTER TABLE nft ADD COLUMN item TEXT")
        except sqlite3.OperationalError: pass
        c.execute("CREATE TABLE IF NOT EXISTS ton_tx(hash TEXT PRIMARY KEY,uid IN
        c.execute("CREATE TABLE IF NOT EXISTS tlog(id INTEGER PRIMARY KEY AUTOINC
        c.execute("CREATE TABLE IF NOT EXISTS ton_lot(id INTEGER PRIMARY KEY AUTO
        c.execute("CREATE TABLE IF NOT EXISTS ton_wd(id INTEGER PRIMARY KEY AUTOI
        c.execute("CREATE TABLE IF NOT EXISTS nft(id INTEGER PRIMARY KEY AUTOINCR
        c.execute("CREATE INDEX IF NOT EXISTS ix_users_bal ON users(bal)"); c.exe
    finally: c.close()

init_db()
_tl = threading.local()

def db():   # у каждого потока своё постоянное соединение (не открываем базу зано
    c = getattr(_tl, "c", None)
    if c is None:
        c = sqlite3.connect(DB_PATH, timeout=15); c.row_factory = sqlite3.Row
        c.execute("PRAGMA synchronous=NORMAL"); c.execute("PRAGMA temp_store=MEMO
        _tl.c = c
    elif c.in_transaction: c.rollback()   # хвост прерванного запроса не должен п
    return c

def _end_tx():
    c = getattr(_tl, "c", None)
    if c is not None and c.in_transaction:
        try: c.rollback()
        except Exception: pass

def auth(init):  # проверка подписи Telegram initData
    try:
        d = dict(urllib.parse.parse_qsl(init or "")); h = d.pop("hash", "")
        s = "\n".join(f"{k}={v}" for k, v in sorted(d.items()))
        key = hmac.new(b"WebAppData", BOT_TOKEN.encode(), hashlib.sha256).digest(
        ok = hmac.compare_digest(hmac.new(key, s.encode(), hashlib.sha256).hexdig
        if ok:
            u = json.loads(d["user"]); return u["id"], u.get("first_name", "")
    except Exception: pass
```

```
    raise HTTPException(401, "Нет доступа")

# ------------------------------------------------------------- как обрабатыва
POOL = ThreadPoolExecutor(max_workers=int(os.environ.get("DB_THREADS", "12")), th
TGPOOL = ThreadPoolExecutor(max_workers=6, thread_name_prefix="tg")
_LAT = deque(maxlen=20000)   # (время, путь, мс) - для /stat в боте

def _resp(res, gz):
    if isinstance(res, Response): return res
    raw = bytes(res) if isinstance(res, (bytes, bytearray)) else json.dumps(res,
    h = {"Cache-Control": "no-store"}
    if gz and len(raw) > 1200:
        raw = gzip.compress(raw, 5); h["Content-Encoding"] = "gzip"; h["Vary"] =
    return Response(raw, media_type="application/json", headers=h)

def api(path, method="post", pool=None):
    """Регистрирует обработчик fn(b): читает JSON, выполняет fn в потоке, отдаёт
    def deco(fn):
        async def h(r: Request):
            p = pool or POOL
            if p._work_queue.qsize() > 400: raise HTTPException(503, "Сервер заня
            try:
                if int(r.headers.get("content-length") or 0) > MAX_BODY: raise HT
            except ValueError: pass
            body = await r.body()
            if len(body) > MAX_BODY: raise HTTPException(413, "Слишком большой за
            gz = "gzip" in r.headers.get("accept-encoding", "")
            t0 = time.perf_counter()
            def work():
                try:
                    try: b = json.loads(body) if body else {}
                    except ValueError: raise HTTPException(400, "Неверный запрос"
                    if not isinstance(b, dict): raise HTTPException(400, "Неверны
                    return _resp(fn(b), gz)
                finally: _end_tx()
            try: return await asyncio.get_running_loop().run_in_executor(p, work)
            finally:
                dt = time.perf_counter() - t0; _LAT.append((time.time(), path, dt
                if dt > 2: log.warning("медленный запрос %s: %.1f c", path, dt)
        h.__name__ = fn.__name__
        getattr(app, method)(path)(h)
        return fn
    return deco

def tg_request(method, params=None, timeout=6):   # вызов Telegram Bot API (тольк
    url = "%s/bot%s/%s" % (TG_API, BOT_TOKEN, method)
    if params is not None: url = urllib.request.Request(url, data=json.dumps(para
```

```
    with urllib.request.urlopen(url, timeout=timeout) as r: return json.load(r)

# ------------------------------------------------------------ сообщения от б
def fnum(n):   # 1 000 000
    try: return "{:,}".format(int(n)).replace(",", " ")
    except Exception: return str(n)

def plate_txt(p):
    try:
        r, d, z, a = str(p.get("r", "")), str(p.get("d", "")), str(p.get("z", "")
        t = ("%s %s %s" % (r, d, z)) if p.get("k") else ("%s %s %s %s" % (r, a, d
        return t + (" ×%s" % p["m"] if p.get("m") else "")
    except Exception: return "номер"

def item_txt(it):
    try:
        v = it.get("v") or {}
        if it.get("t") == "car":
            return "🚗 %s%s" % (str(v.get("n", "машина"))[:40], (" + номер " + pl
        return "🔢 номер " + plate_txt(v)
    except Exception: return "предмет"

def left_txt(sec):
    sec = max(0, int(sec))
    if sec >= 3600: return "%d ч %d мин" % (sec // 3600, sec % 3600 // 60)
    if sec >= 60: return "%d мин" % (sec // 60)
    return "%d сек" % sec

def notify(uid, text, btn="🎰 Открыть игру"):
    """Отправить игроку сообщение от бота (в фоне; если игрок заблокировал бота -
    try: uid = int(uid)
    except (TypeError, ValueError): return
    if uid <= 0 or TGPOOL._work_queue.qsize() > 3000: return
    def job():
        try:
            m = {"chat_id": uid, "text": text[:3500], "disable_web_page_preview":
            if APP_URL.startswith("https://"): m["reply_markup"] = {"inline_keybo
            tg_request("sendMessage", m, timeout=8)
        except Exception as e: log.debug("сообщение игроку %s не доставлено: %s",
    try: TGPOOL.submit(job)
    except Exception: pass

# ------------------------------------------------------------ единственные э
SPSER = {"UZB", "AAA", "SBA"}
SPX = [(1, "879", "MEA", ""), (1, "762", "MMA", ""), (1, "500", "SOO", ""), (0, "
SSX = {"PAA": 4, "PSS": 4, "PPP": 3, "APC": 3, "TSS": 3, "TST": 3, "ZDF": 3, "DXA
       "TTA": 1, "TTB": 1, "TTC": 1, "TTD": 1, "TTE": 1, "TTF": 1, "TTG": 1}
```

```
def tier_py(p):  # то же, что tier() в игре
    try: k = int(p["k"]); r = str(p["r"]); d = str(p["d"]); z = str(p["z"]); a =
    except Exception: return 0
    if r == "01" and any(x[0] == k and x[1] == d and x[2] == z and x[3] == a for
    if k and (z in SPSER or (r == "01" and d == "007" and z == "AUB")): return 5
    if k and z in SSX: return SSX[z]
    if len(d) != 3 or len(z) < 2: return 0
    sc = 0
    if d[0] == d[1] == d[2]: sc += 4
    elif d in ("123", "321", "456", "789", "007", "100") or d[1:] == "00": sc +=
    elif d[0] == d[2]: sc += 1
    if z[0] == z[1]: sc += 1
    if len(z) > 2 and z[1] == z[2]: sc += 1
    if r == "01": sc += 1
    return 4 if sc >= 6 else 3 if sc >= 5 else 2 if sc >= 3 else 1 if sc >= 2 els

def ukey(p): return "%s|%s|%s|%s|%s" % (p.get("k"), p.get("r"), p.get("a") or "",
def is_uniq(p): return isinstance(p, dict) and tier_py(p) >= 4

def need_own(c, uid, v):
    """Единственный номер (Мифик/Секретный) можно выставить/поставить в дуэль тол
    иначе можно подделать чужой или несуществующий номер."""
    pl = v.get("pl") if isinstance(v, dict) and "pl" in v else v
    if not isinstance(pl, dict): return
    try: p = clean_plate(pl)
    except HTTPException: raise HTTPException(400, "Неверный номер")
    if tier_py(p) >= 4:
        row = c.execute("SELECT owner FROM uniq WHERE key=?", (ukey(p),)).fetchon
        if not row or row["owner"] != uid: raise HTTPException(400, "Этот единств

def uniq_set(c, uid, p):   # передать номер новому владельцу
    c.execute("INSERT OR REPLACE INTO uniq(key,owner,ts) VALUES(?,?,?)", (ukey(p)

_GIFTED = {}   # когда игрок последний раз получал что-то от сервера (подарок, пр
def gift(c, uid, d):
    _GIFTED[uid] = time.time()
    try:   # тюнинг переезжает вместе с машиной к новому владельцу; подделанные i
        if d.get("t") == "car" and isinstance(d.get("v"), dict):
            v = d["v"]; cid = v.get("id")
            if cid:
                r = c.execute("SELECT uid,s,su,ti,li,ae,ni FROM car_tune WHERE ci
                if r:
                    if r["uid"] != uid:
                        c.execute("DELETE FROM car_tune WHERE uid=? AND cid=?", (
                        c.execute("UPDATE car_tune SET uid=? WHERE uid=? AND cid=
                    v["tn"] = [r["s"], r["su"], r["ti"], r["li"], r["ae"] or 0, r
```

```
                else: v.pop("id", None); v.pop("tn", None)
            else: v.pop("tn", None)
    except Exception: log.exception("gift tune")
    c.execute("INSERT INTO gifts(uid,data) VALUES(?,?)", (uid, json.dumps(d)))
    try:   # номер/машина с номером: единственный экземпляр переходит получателю
        if d.get("t") == "plate" and is_uniq(d.get("v")): uniq_set(c, uid, d["v"]
        elif d.get("t") == "car" and is_uniq((d.get("v") or {}).get("pl")): uniq_
    except Exception: pass

UPG_X = (2, 5, 10); UPG_RTP = 0.94; UPG_INS = 29
UPG_MAXM = 32                                    # номер можно улучшать снова и сн
def m_ok(m):                                        # допустимый множитель номера:
    if not isinstance(m, int) or m < 2 or m > UPG_MAXM: return False
    for q in (2, 5):
        while m % q == 0: m //= q
    return m == 1
def clean_plate(p):
    try: k = int(p["k"]); r = str(p["r"]); d = str(p["d"]); z = str(p["z"]).upper
    except Exception: raise HTTPException(400, "Неверный номер")
    o = {"k": k, "r": r, "d": d, "z": z, **({"a": a} if k == 0 else {})}
    try: m = int(p.get("m") or 0)
    except (TypeError, ValueError): m = 0
    if m_ok(m): o["m"] = m
    return o

# ---- защита от накрутки: суточные лимиты на действия, которые создают ценности
UCL_DAY = int(os.environ.get("UCL_DAY", 5))        # новых Мифик/Секретных номеро
DUEL_DAY = int(os.environ.get("DUEL_DAY", 10))     # дуэлей (создать/принять) в с
OFFER_DAY = int(os.environ.get("OFFER_DAY", 50 * 10 ** 9))   # сумма продаж обмен
OFFER_ALERT = int(os.environ.get("OFFER_ALERT", 5 * 10 ** 9))
_LIM = {}
def lim_hit(kind, uid, n=1, cap=1):   # True - лимит исчерпан (счётчик не растёт)
    k = (kind, uid); t = time.time(); w = _LIM.get(k)
    if not w or t - w[0] > 86400: w = [t, 0]
    if w[1] + n > cap: _LIM[k] = w; return True
    w[1] += n; _LIM[k] = w
    if len(_LIM) > 200000: _LIM.clear()
    return False
def alert_admins(txt):
    for a in HIDDEN: notify(a, txt, "🎰 Открыть игру")

@api("/api/uclaim")
def uclaim(b):   # игрок выбил номер: если это Мифик/Секретный - закрепить его за
    uid, _ = auth(b.get("initData")); p = clean_plate(b.get("p") or {}); c = db()
    if tier_py(p) < 4: return {"ok": 1, "free": 1}
    c.execute("INSERT OR IGNORE INTO uniq(key,owner,ts) VALUES(?,?,?)", (ukey(p),
    new = c.execute("SELECT changes()").fetchone()[0] == 1
```

```
    if new and uid not in HIDDEN and lim_hit("ucl", uid, 1, UCL_DAY):   # слишком
        c.execute("DELETE FROM uniq WHERE key=? AND owner=?", (ukey(p), uid)); c.
        log.warning("ПОДОЗРЕНИЕ: %s превысил суточный лимит редких номеров", uid)
        if not lim_hit("ucl_al", uid, 1, 1): alert_admins("⚠️ Игрок ID %s пытаетс
        return {"ok": 0, "taken": 1}
    c.commit()
    row = c.execute("SELECT owner FROM uniq WHERE key=?", (ukey(p),)).fetchone()
    if row and row["owner"] != uid: return {"ok": 0, "taken": 1}
    try: drop_alert(c, uid, p)
    except Exception: log.exception("drop_alert")
    return {"ok": 1}

@api("/api/urel")
def urel(b):   # игрок продал уникальный номер магазину - он снова может выпасть
    uid, _ = auth(b.get("initData")); p = clean_plate(b.get("p") or {}); c = db()
    c.execute("UPDATE uniq SET owner=0 WHERE key=? AND owner=?", (ukey(p), uid));
    return {"ok": 1}

@api("/api/ucheck")
def ucheck(b):   # проверить номер: редкость, занят ли
    uid, _ = auth(b.get("initData")); p = clean_plate(b.get("p") or {}); c = db()
    row = c.execute("SELECT owner FROM uniq WHERE key=?", (ukey(p),)).fetchone()
    own = None
    if row:
        u = c.execute("SELECT name FROM users WHERE id=?", (row["owner"],)).fetch
    return {"tier": t, "unique": t >= 4, "taken": bool(row), "owner": own}

@api("/api/ustats", method="get")
def ustats(b):
    c = db(); n = {4: 0, 5: 0}
    for x in c.execute("SELECT key FROM uniq").fetchall():
        k, r, a, d, z = x["key"].split("|"); t = tier_py({"k": int(k), "r": r, "a
        if t in n: n[t] += 1
    return {"myth": n[4], "secret": n[5]}

def uniq_backfill():   # один раз при запуске: записать за игроками их Мифик/Секр
    c = db()
    if c.execute("SELECT 1 FROM meta WHERE k='uniq_bf'").fetchone(): return
    for (uid,) in c.execute("SELECT id FROM users WHERE state IS NOT NULL").fetch
        row = c.execute("SELECT state FROM users WHERE id=?", (uid,)).fetchone()
        try: st = json.loads(row["state"])
        except Exception: continue
        pls = list(st.get("col") or []) + [x.get("pl") for x in (st.get("cars") o
        for p in pls:
            if is_uniq(p) and not c.execute("SELECT 1 FROM uniq WHERE key=?", (uk
    c.execute("INSERT OR REPLACE INTO meta(k,v) VALUES('uniq_bf','1')"); c.commit
```

```
# ------------------------------------------------------------------ главная страни
_IDX = {}
def _index_entry():
    path = os.path.join(HERE, "uz-nomerok-tg.html"); mt = os.stat(path).st_mtime_
    e = _IDX.get("e")
    if e and e["mt"] == mt: return e
    raw = open(path, "rb").read()
    e = {"mt": mt, "raw": raw, "gz": gzip.compress(raw, 9), "etag": '"%s"' % hash
    _IDX["e"] = e; return e

@app.get("/")
async def index(r: Request):
    e = _index_entry()
    h = {"ETag": e["etag"], "Cache-Control": "no-cache", "Vary": "Accept-Encoding
    if r.headers.get("if-none-match") == e["etag"]: return Response(status_code=3
    if "gzip" in r.headers.get("accept-encoding", ""):
        h["Content-Encoding"] = "gzip"; return Response(e["gz"], media_type="text
    return Response(e["raw"], media_type="text/html; charset=utf-8", headers=h)

@app.get("/healthz")
async def healthz(): return Response('{"ok":1}', media_type="application/json", h

# ------------------------------------------------------------------ общие запросы
def get_rid(c):  # номер "сезона": при сбросе прогресса увеличивается
    r = c.execute("SELECT v FROM meta WHERE k='rid'").fetchone()
    return int(r[0]) if r else 0

def gifts_of(c, uid):
    return [{"id": x["id"], **json.loads(x["data"])} for x in c.execute("SELECT i

def ref_count(c, uid): return c.execute("SELECT COUNT(*) FROM referrals WHERE ref

def offers_of(c, uid):
    def rows(w): return [{"id": x["id"], "frm": x["frm"], "to_id": x["to_id"], "i
                         for x in c.execute("SELECT * FROM offers WHERE status='o
    return {"incoming": rows("to_id=?"), "outgoing": rows("frm=?")}

_EVC = [0, None]
def event_of(c):   # текущий ивент (кэш 5 сек), общий для всех игроков
    t = time.time()
    if _EVC[1] is None or t - _EVC[0] > 5:
        e = c.execute("SELECT * FROM events WHERE until>? ORDER BY until DESC LIM
        _EVC[0] = t; _EVC[1] = {**(dict(e) if e else {}), "rid": get_rid(c)}
    return _EVC[1]

def ref_credit(c, uid):  # засчитать приглашение, когда новый игрок впервые зашёл
    p = c.execute("SELECT ref FROM refpending WHERE uid=?", (uid,)).fetchone()
```

```
    if not p or p["ref"] == uid or not c.execute("SELECT 1 FROM users WHERE id=?"
    if c.execute("INSERT OR IGNORE INTO referrals VALUES(?,?)", (uid, p["ref"])).
    n = c.execute("SELECT COUNT(*) FROM referrals WHERE ref=?", (p["ref"],)).fetc
    for need, reward in REF_MILESTONES:
        if n == need: gift(c, p["ref"], {"t": "bal", "v": reward})
    if n == REF_LUCK[0]: gift(c, p["ref"], {"t": "boost", "k": "bl", "v": REF_LUC

def _bal_of(st):
    try: v = int(float(st.get("bal") or 0))
    except Exception: v = 0
    return max(0, min(v, 9 * 10 ** 15))

def _hidden_ids():   # админы (ADMINS) и HIDE_IDS не показываются в рейтинге
    out = set()
    for x in (os.environ.get("ADMINS", "") + "," + os.environ.get("HIDE_IDS", "")
        if x.strip().lstrip("-").isdigit(): out.add(int(x))
    return out
HIDDEN = _hidden_ids()
BAL_ROLL = int(os.environ.get("BAL_ROLL", 10 ** 10))   # рост баланса без причины
BAL_JUMP = int(os.environ.get("BAL_JUMP", 3 * 10 ** 9))   # скачок баланса за одн
_JUMP = {}

def bal_backfill():   # один раз: заполнить рейтинг по уже сохранённым игрокам
    c = db()
    if c.execute("SELECT 1 FROM meta WHERE k='bal_bf'").fetchone(): return
    for r in c.execute("SELECT id,state FROM users WHERE state IS NOT NULL").fetc
        try: c.execute("UPDATE users SET bal=? WHERE id=?", (_bal_of(json.loads(r
        except Exception: pass
    c.execute("INSERT OR REPLACE INTO meta(k,v) VALUES('bal_bf','1')"); c.commit(

def bibisha_purge():   # один раз: убрать всех Бибиш у всех игроков (теперь выдаю
    c = db()
    if c.execute("SELECT 1 FROM meta WHERE k='bibisha_purge'").fetchone(): return
    n = 0
    for r in c.execute("SELECT id,state FROM users WHERE state IS NOT NULL").fetc
        try:
            st = json.loads(r["state"]); L = st.get("cars") or []
            nl = [x for x in L if not (isinstance(x, dict) and "Бибиша" in str(x.
            if len(nl) != len(L): st["cars"] = nl; c.execute("UPDATE users SET st
        except Exception: pass
    c.execute("INSERT OR REPLACE INTO meta(k,v) VALUES('bibisha_purge','1')"); c.

def jump_why(old_json, st, last_seen):   # что изменилось в состоянии игрока межд
    try: o = json.loads(old_json) if old_json else {}
    except Exception: o = {}
    lines = []; dt = int(time.time()) - int(last_seen or 0)
    if last_seen: lines.append("С прошлого сохранения прошло: %s" % (("%d мин" %
```

```
    ch = []
    for k, v in st.items():
        if k in ("bal", "rid") or isinstance(v, bool) or not isinstance(v, (int,
        ov = o.get(k)
        if isinstance(ov, (int, float)) and not isinstance(ov, bool) and ov != v
    if ch: lines.append("Изменились счётчики: " + "; ".join(ch[:12]))
    for k, nm in (("col", "номера в гараже"), ("cars", "машины")):
        a, b = o.get(k), st.get(k)
        if isinstance(a, list) and isinstance(b, list) and len(a) != len(b): line
    try:
        seen = {json.dumps(x, sort_keys=True) for x in (o.get("col") or [])}
        new = [x for x in (st.get("col") or []) if isinstance(x, dict) and json.d
        if new: lines.append("Новые номера: " + ", ".join(plate_txt(x) for x in n
    except Exception: pass
    sp, sold = (st.get("sp") or 0) - (o.get("sp") or 0), (st.get("sold") or 0) -
    if sp <= 0 and sold <= 0 and (st.get("bpx") or 0) == (o.get("bpx") or 0): lin
    elif sold > 0: lines.append("Продаж номеров за это время: %d — возможно, чест
    else: lines.append("Круток за это время: %d" % max(sp, 0))
    return "\n".join(lines)

UQ_STATE = {0: "Выведен из игры", -1: "Выставлен на аукцион за ⭐", -2: "Выводитс

@api("/api/plate/find")
def plate_find(b):
    """Поиск единственного номера (Мифик / Секретный): кто владелец сейчас и скол
    uid, _ = auth(b.get("initData")); c = db()
    p = clean_plate(b.get("plate") or {})
    if not (p["d"].isdigit() and len(p["d"]) == 3 and 2 <= len(p["z"]) <= 3 and l
    t = tier_py(p)
    out = {"plate": {k: v for k, v in p.items() if k != "m"}, "tier": t}
    if t < 4: return {**out, "found": 0}
    k = ukey(p); row = c.execute("SELECT owner FROM uniq WHERE key=?", (k,)).fetc
    if not row: return {**out, "found": 1, "free": 1, "owners": 0}
    n = c.execute("SELECT COUNT(*) FROM uniq_own WHERE key=?", (k,)).fetchone()[0
    o = row["owner"]; res = {**out, "found": 1, "owners": n}
    if o > 0:
        u = c.execute("SELECT name FROM users WHERE id=?", (o,)).fetchone()
        res["owner"] = {"id": o, "name": ((u["name"] if u else "") or "Игрок")[:2
    else: res["state"] = UQ_STATE.get(o, "В игре")
    return res

_TOP = {"ts": 0, "list": []}
@api("/api/top")
def top(b):
    uid, _ = auth(b.get("initData")); c = db(); now = time.time()
    if now - _TOP["ts"] > 15:
        hid = sorted(HIDDEN); q = ",".join("?" * len(hid))
```

```
        rows = c.execute("SELECT id,name,bal FROM users WHERE bal>0 " + ("AND id
        _TOP["list"] = [{"id": r["id"], "name": (r["name"] or "Игрок")[:18], "bal
    out = [{"n": i + 1, "name": x["name"], "bal": x["bal"], "me": x["id"] == uid}
    me = None
    if uid not in HIDDEN:
        r = c.execute("SELECT bal FROM users WHERE id=?", (uid,)).fetchone(); mb
        hid = sorted(HIDDEN); q = ",".join("?" * len(hid))
        rk = c.execute("SELECT COUNT(*) FROM users WHERE bal>?" + (" AND id NOT I
        me = {"rank": rk, "bal": mb}
    return {"list": out, "me": me}

def wallet_of(c, uid):
    r = c.execute("SELECT COALESCE(stars,0) FROM users WHERE id=?", (uid,)).fetch
    return int(r[0]) if r else 0

def wadd(c, uid, n, why):
    c.execute("UPDATE users SET stars=COALESCE(stars,0)+? WHERE id=?", (int(n), u
    c.execute("INSERT INTO wlog(uid,d,why,ts) VALUES(?,?,?,?)", (uid, int(n), why

def wspend(c, uid, n, why):   # списать звёзды атомарно: не получится - ничего не
    n = int(n)
    if n <= 0 or c.execute("UPDATE users SET stars=stars-? WHERE id=? AND COALESC
        c.rollback(); raise HTTPException(400, "Не хватает ⭐ на звёздном балансе
    c.execute("INSERT INTO wlog(uid,d,why,ts) VALUES(?,?,?,?)", (uid, -n, why, in

def stars_of(c, uid): return wallet_of(c, uid)

@api("/api/load")
def load(b):
    uid, name = auth(b.get("initData")); now = int(time.time()); c = db()
    existed = c.execute("SELECT 1 FROM users WHERE id=?", (uid,)).fetchone()
    c.execute("INSERT INTO users(id,name,first_seen,last_seen) VALUES(?,?,?,?) ON
    if not existed: ref_credit(c, uid)
    row = c.execute("SELECT state FROM users WHERE id=?", (uid,)).fetchone()
    head = json.dumps({"uid": uid, "gifts": gifts_of(c, uid), "ref": {"n": ref_co
    c.commit()
    state = row["state"] if row and row["state"] else "null"   # состояние уже хр
    if state != "null" and ('"z"' in state):
        try:
            st = json.loads(state); uniq_guard(c, uid, st); st.pop("_own", None)
            ns = json.dumps(st, separators=(",", ":"))
            if ns != state:
                c.execute("UPDATE users SET state=?,bal=? WHERE id=?", (ns, _bal_
        except Exception: log.exception("load: чистка дублей")
    return ('{"state":%s,%s' % (state, head[1:])).encode()

_SEEN = {}
```

```
@api("/api/poll")
def poll(b):   # один лёгкий запрос вместо четырёх: подарки, обмены, друзья, ивен
    uid, _ = auth(b.get("initData")); c = db(); t = time.time()
    if t - _SEEN.get(uid, 0) > 120:
        _SEEN[uid] = t
        if len(_SEEN) > 100000: _SEEN.clear()
        c.execute("UPDATE users SET last_seen=? WHERE id=?", (int(t), uid)); c.co
    return {"gifts": gifts_of(c, uid), "offers": offers_of(c, uid), "ref": {"n":

def uniq_guard(c, uid, st):
    """Единственные номера (Мифик/Секретный): у каждого - ровно один владелец.
    Чужой/уже занятый номер в состоянии игрока удаляется, свободный - закрепляетс
    dups = []; seen = set(); st["_own"] = 0
    def chk(p):
        if not isinstance(p, dict): return True
        try: pp = clean_plate(p)
        except HTTPException: return True
        if tier_py(pp) < 4: return True
        k = ukey(pp)
        c.execute("INSERT OR IGNORE INTO uniq(key,owner,ts) VALUES(?,?,?)", (k, u
        if c.execute("SELECT changes()").fetchone()[0] == 1 and uid not in HIDDEN
            c.execute("DELETE FROM uniq WHERE key=? AND owner=?", (k, uid)); dups
            if not lim_hit("ucl_al", uid, 1, 1): alert_admins("⚠️ Игрок ID %s: в
            return False
        row = c.execute("SELECT owner FROM uniq WHERE key=?", (k,)).fetchone()
        if row and row["owner"] != uid:
            dups.append(pp); return False
        if k in seen:            # второй такой же номер у самого владельца
            st["_own"] += 1; return False
        seen.add(k)
        return True
    col = st.get("col")
    if isinstance(col, list):
        st["col"] = [p for p in col if chk(p)]
    cars = st.get("cars")
    if isinstance(cars, list):
        for car in cars:
            if isinstance(car, dict) and car.get("pl") is not None and not chk(ca
    if dups:
        log.warning("uniq: дубликаты у %s: %s", uid, [ukey(p) for p in dups])
    if st.get("_own"): log.warning("uniq: двойники у владельца %s: %s", uid, st["
    return dups

@api("/api/save")
def save(b):
    uid, _ = auth(b.get("initData")); c = db(); st = b.get("state")
    if not isinstance(st, dict): raise HTTPException(400, "Неверное состояние")
```

```
    if str(st.get("rid", 0)) != str(get_rid(c)): return {"stale": 1}   # игра из
    if not st.get("cl2") and isinstance(st.get("cars"), list):   # старый клиент:
        st["cars"] = [x for x in st["cars"] if not (isinstance(x, dict) and "Биби
    try:   # недавние апгрейды: старое сохранение не должно вернуть сгоревший ном
        c.execute("DELETE FROM upg_pend WHERE ts<?", (int(time.time()) - 120,))
        for r_ in c.execute("SELECT pk,win,newm,cnt FROM upg_pend WHERE uid=? ORD
            d_ = json.loads(r_["pk"])
            if r_["win"] == -2: _tune_pend(st, d_)
            elif r_["win"] == -1:
                if d_.get("t") == "plate":
                    pp = clean_plate(d_["v"])
                    while _count_pl(st, pp) > (r_["cnt"] or 0): _strip_item(st, d
                else: _strip_item(st, d_)
            else:
                while _count_pl(st, d_) > (r_["cnt"] or 0):
                    if not _upg_apply(st, d_, r_["win"], r_["newm"]): break
    except Exception: log.exception("upg_pend")
    try: _tune_apply(c, uid, st)
    except Exception: log.exception("tune_apply")
    dups = uniq_guard(c, uid, st); own = st.pop("_own", 0); rb = None
    try:
        o0 = c.execute("SELECT bal,state FROM users WHERE id=?", (uid,)).fetchone
        if o0 and uid not in HIDDEN and o0["state"]:
            ob, nb0 = (o0["bal"] or 0), _bal_of(st)
            if nb0 - ob > BAL_ROLL and time.time() - _GIFTED.get(uid, 0) > 900:
                os_ = json.loads(o0["state"])
                same = all((st.get(k) or 0) == (os_.get(k) or 0) for k in ("sp",
                same = same and all(isinstance(st.get(k), list) == isinstance(os_
                if same:   # деньги выросли, а ничего не крутили, не продавали и
                    log.warning("ОТКАТ баланса %s: %s -> %s", uid, ob, nb0); st["
                    if not lim_hit("rb_al", uid, 1, 1): alert_admins("🛑 Накрутка
    except Exception: log.exception("rollback check")
    try:    # подозрительный скачок баланса за одно сохранение - сообщаем админам
        old = c.execute("SELECT bal,state,last_seen FROM users WHERE id=?", (uid,
        if old and nb - (old["bal"] or 0) > BAL_JUMP and uid not in HIDDEN and ti
            _JUMP[uid] = time.time(); log.warning("ПОДОЗРЕНИЕ: баланс %s: %s -> %
            why = jump_why(old["state"], st, old["last_seen"])
            for a in HIDDEN: notify(a, "⚠️ Подозрительный скачок баланса\nИгрок I
    except Exception: log.exception("jump check")
    c.execute("UPDATE users SET state=?,last_seen=?,bal=? WHERE id=?", (json.dump
    r = {"ok": 1}
    if rb is not None: r["rollback"] = 1; r["bal"] = rb
    if dups: r["dup"] = dups
    if own: r["own"] = own
    return r

@api("/api/gifts/clear")
```

```
def clear(b):
    uid, _ = auth(b.get("initData")); c = db()
    try: ids = [int(i) for i in (b.get("ids") or [])][:500]
    except (TypeError, ValueError): raise HTTPException(400, "Неверные данные")
    if ids: c.execute("DELETE FROM gifts WHERE uid=? AND id IN (%s)" % ",".join("
    return {"ok": 1}

@api("/api/gifts")
def gifts(b):
    uid, _ = auth(b.get("initData")); return {"gifts": gifts_of(db(), uid)}

@api("/api/promo")
def promo(b):
    uid, _ = auth(b.get("initData")); code = str(b.get("code", "")).strip().upper
    p = c.execute("SELECT * FROM promos WHERE code=?", (code,)).fetchone()
    if not p: raise HTTPException(400, "Промокод не найден")
    if p["used"] >= p["max_uses"]: raise HTTPException(400, "Промокод закончился"
    try: c.execute("INSERT INTO redeemed VALUES(?,?)", (code, uid))
    except sqlite3.IntegrityError: raise HTTPException(400, "Вы уже активировали
    if c.execute("UPDATE promos SET used=used+1 WHERE code=? AND used<max_uses",
        c.rollback(); raise HTTPException(400, "Промокод закончился")
    c.commit()
    _GIFTED[uid] = time.time()
    return {"reward": json.loads(p["reward"])}

@api("/api/offer/create")
def offer_create(b):
    uid, uname = auth(b.get("initData")); c = db()
    try: to = int(b["to"]); price = int(b["price"])
    except (KeyError, TypeError, ValueError): raise HTTPException(400, "Неверные
    item = b.get("item")
    if price < 0 or not c.execute("SELECT 1 FROM users WHERE id=?", (to,)).fetcho
        raise HTTPException(400, "Игрок с таким ID не найден")
    if not isinstance(item, dict) or item.get("t") not in ("plate", "car") or not
        raise HTTPException(400, "Неверные данные предложения")
    need_own(c, uid, item["v"])
    if price > OFFER_DAY: raise HTTPException(400, "Слишком высокая цена")
    c.execute("INSERT INTO offers(frm,to_id,item,price,ts) VALUES(?,?,?,?,?)", (u
    notify(to, "🤝 %s предлагает вам обмен:\n%s\nЦена: %s" % (str(uname)[:30] or
    return {"ok": 1}

@api("/api/offer/list")
def offer_list(b):
    uid, _ = auth(b.get("initData")); return offers_of(db(), uid)

@api("/api/offer/accept")
def offer_accept(b):
```

```
    uid, _ = auth(b.get("initData")); c = db()
    o = c.execute("SELECT * FROM offers WHERE id=? AND status='open'", (int(b.get
    if not o or o["to_id"] != uid: raise HTTPException(400, "Предложение недоступ
    if c.execute("UPDATE offers SET status='done' WHERE id=? AND status='open'",
        raise HTTPException(400, "Предложение недоступно")
    if o["price"] > 0 and o["frm"] not in HIDDEN and lim_hit("offer", o["frm"], o
        c.execute("UPDATE offers SET status='open' WHERE id=?", (o["id"],)); c.co
    if o["price"] >= OFFER_ALERT: alert_admins("⚠️ Крупная сделка обменом: ID %s
    gift(c, uid, json.loads(o["item"])); gift(c, o["frm"], {"t": "bal", "v": o["p
    notify(o["frm"], "✅ Ваше предложение принято!\n%s\nВам начислено: %s сум" %
    return {"ok": 1}

@api("/api/offer/cancel")
def offer_cancel(b):
    uid, _ = auth(b.get("initData")); c = db()
    o = c.execute("SELECT * FROM offers WHERE id=? AND status='open'", (int(b.get
    if not o or o["frm"] != uid: raise HTTPException(400, "Предложение недоступно
    if c.execute("UPDATE offers SET status='cancelled' WHERE id=? AND status='ope
        raise HTTPException(400, "Предложение недоступно")
    gift(c, uid, json.loads(o["item"])); c.commit()
    return {"ok": 1}

# ---------------------------------------------------------------- NFT (TON): кош
import re as _re, html as _html
TON_RE = _re.compile(r"^((EQ|UQ|kQ|0Q)[A-Za-z0-9_-]{46}|-?[0-9]:[0-9a-fA-F]{64})$
NFT_NAMES = {4: "Мифик", 5: "Секретный"}

def nft_title(p): return ("%s %s %s" % (p["d"], p["z"], p["r"])) if p["k"] else (

@api("/api/wallet")
def wallet_api(b):
    uid, _ = auth(b.get("initData")); c = db(); addr = (b.get("addr") or "").stri
    if addr:
        if not TON_RE.match(addr): raise HTTPException(400, "Неверный адрес TON-к
        c.execute("INSERT OR REPLACE INTO wallets(uid,addr,ts) VALUES(?,?,?)", (u
    r = c.execute("SELECT addr FROM wallets WHERE uid=?", (uid,)).fetchone()
    return {"ok": 1, "addr": r["addr"] if r else ""}

@api("/api/nft/req")
def nft_req(b):
    """Вывести Мифик/Секретный номер в NFT: номер блокируется (escrow), админ вып
    uid, _ = auth(b.get("initData")); c = db(); p = clean_plate(b.get("p") or {})
    if tier_py(p) < 4: raise HTTPException(400, "В NFT можно вывести только Мифик
    w = c.execute("SELECT addr FROM wallets WHERE uid=?", (uid,)).fetchone()
    if not w: raise HTTPException(400, "Сначала подключите TON-кошелёк")
    if c.execute("SELECT COUNT(*) FROM nft WHERE uid=? AND status='pending'", (ui
    if c.execute("UPDATE uniq SET owner=-2 WHERE key=? AND owner=?", (ukey(p), ui
```

```
        c.rollback(); raise HTTPException(400, "Этот единственный номер вам не пр
    try: cur = c.execute("INSERT INTO nft(uid,key,plate,addr,ts) VALUES(?,?,?,?,?
    except sqlite3.IntegrityError:
        c.rollback(); raise HTTPException(400, "Заявка на этот номер уже есть")
    c.commit(); nid = cur.lastrowid
    for a in ADMIN_IDS:
        notify(a, "💎 Заявка на NFT #%d: %s (%s)\nИгрок %s, кошелёк %s\n/nftdone
    return {"ok": 1, "id": nid}

@api("/api/nft/my")
def nft_my(b):
    uid, _ = auth(b.get("initData")); c = db()
    return {"list": [{"id": r["id"], "plate": json.loads(r["plate"]), "status": r

def nft_svg(p):
    t = tier_py(p); col = "#ec4899" if t >= 5 else "#f59e0b"
    num = _html.escape(("%s  %s" % (p["d"], p["z"])) if p["k"] else ("%s %s %s" %
    return ('<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 600 600"><defs>
            '<rect width="600" height="600" fill="url(#g)"/><rect x="40" y="215"
            '<rect x="40" y="215" width="120" height="170" rx="22" fill="#e5e7eb"
            '<text x="370" y="332" font-family="Arial" font-weight="900" font-siz
            '<text x="300" y="150" font-family="Arial" font-weight="900" font-siz
            '<text x="300" y="470" font-family="Arial" font-size="26" text-anchor

@app.get("/nft/{name}")
async def nft_file(r: Request):
    name = r.path_params["name"]; base = APP_URL.rstrip("/")
    if name == "collection.json":
        return Response(json.dumps({"name": "UZ Номерок", "description": "Редкие
    if name == "icon.svg":
        return Response(nft_svg({"k": 1, "r": "01", "d": "777", "z": "UZB"}), med
    m = _re.match(r"^(\d+)\.(json|svg)$", name)
    if not m: raise HTTPException(404, "нет")
    row = db().execute("SELECT plate FROM nft WHERE id=?", (int(m.group(1)),)).fe
    if not row: raise HTTPException(404, "нет")
    p = json.loads(row["plate"])
    if m.group(2) == "svg": return Response(nft_svg(p), media_type="image/svg+xml
    return Response(json.dumps({"name": "%s" % nft_title(p), "description": "%s н
        "image": "%s/nft/%s.svg" % (base, m.group(1)), "attributes": [{"trait_typ

@app.get("/tonconnect-manifest.json")
def tc_manifest():
    base = APP_URL.rstrip("/")
    return Response(json.dumps({"url": base, "name": "UZ Номерок", "iconUrl": bas

# ---------------------------------------------------------------- TON: пополнени
import base64 as _b64, decimal as _dec
```

```
TON_WALLET = os.environ.get("TON_WALLET", "").strip()                          #
TON_API = os.environ.get("TON_API", "https://toncenter.com/api/v3").rstrip("/")
TON_KEY = os.environ.get("TON_API_KEY", "")
NANO = 10 ** 9; TON_FEE = 5; TON_MIN_DEP = NANO // 20; TON_MIN_WD = NANO // 2; TO
try: TON_STAR_RATE = max(1, int(os.environ.get("TON_STAR_RATE", "150")))      # c
except ValueError: TON_STAR_RATE = 150

def raw_addr(a):
    """любой формат адреса TON -> 'wc:hex' (для сравнения)"""
    a = (a or "").strip()
    if _re.match(r"^-?[0-9]:[0-9a-fA-F]{64}$", a): return a.lower()
    try:
        b = _b64.urlsafe_b64decode(a + "=" * (-len(a) % 4))
        if len(b) == 36: return "%d:%s" % (b[1] if b[1] < 128 else b[1] - 256, b[
    except Exception: pass
    return ""

def _crc16(d):
    c = 0
    for x in d:
        c ^= x << 8
        for _ in range(8): c = ((c << 1) ^ 0x1021) & 0xFFFF if c & 0x8000 else (c
    return c

def friendly_addr(raw):   # 'wc:hex' -> UQ... (не bounceable)
    try:
        wc, h = raw.split(":"); b = bytes([0x51, int(wc) & 0xFF]) + bytes.fromhex
        return _b64.urlsafe_b64encode(b).decode()
    except Exception: return raw

def ton_get(path, params):
    url = TON_API + path + "?" + urllib.parse.urlencode(params)
    req = urllib.request.Request(url, headers={"X-API-Key": TON_KEY} if TON_KEY e
    with urllib.request.urlopen(req, timeout=10) as r: return json.load(r)

def ton_of(c, uid):
    r = c.execute("SELECT COALESCE(ton,0) FROM users WHERE id=?", (uid,)).fetchon
def tadd(c, uid, n, why):
    c.execute("UPDATE users SET ton=COALESCE(ton,0)+? WHERE id=?", (int(n), uid))
def tspend(c, uid, n, why):
    n = int(n)
    if n <= 0 or c.execute("UPDATE users SET ton=ton-? WHERE id=? AND COALESCE(to
        c.rollback(); raise HTTPException(400, "Не хватает TON на балансе")
    c.execute("INSERT INTO tlog(uid,d,why,ts) VALUES(?,?,?,?)", (uid, -n, why, in

_TP = [0.0]; _TLOCK = threading.Lock()
def ton_poll(force=False):
```

```
    """Забрать входящие переводы на кошелёк игры и зачислить те, где в комментари
    if not TON_WALLET: return 0
    if not _TLOCK.acquire(blocking=False): return 0
    try:
        if not force and time.time() - _TP[0] < 8: return 0
        _TP[0] = time.time(); n = 0
        d = ton_get("/transactions", {"account": TON_WALLET, "limit": 40, "sort":
        for tx in d.get("transactions", []):
            m = tx.get("in_msg") or {}; src = m.get("source")
            if not src: continue
            try: val = int(m.get("value") or 0)
            except (TypeError, ValueError): continue
            cm = ((m.get("message_content") or {}).get("decoded") or {}).get("com
            mm = _re.match(r"^\s*uz(\d+)\s*$", cm)
            if not mm or val < TON_MIN_DEP or not tx.get("hash"): continue
            if ((tx.get("description") or {}).get("aborted")): continue
            uid = int(mm.group(1))
            if not c.execute("SELECT 1 FROM users WHERE id=?", (uid,)).fetchone()
            if c.execute("INSERT OR IGNORE INTO ton_tx(hash,uid,amount,src,ts) VA
            tadd(c, uid, val, "deposit"); sr = friendly_addr(raw_addr(src)) or sr
            c.execute("INSERT OR REPLACE INTO wallets(uid,addr,ts,ver) VALUES(?,?
            notify(uid, "💎 Зачислено %.2f TON на ваш баланс" % (val / NANO))
        return n
    finally: _TLOCK.release()

def ton_fee(price): return price * TON_FEE // 100

@api("/api/ton/info")
def ton_info(b):
    uid, _ = auth(b.get("initData")); c = db(); w = c.execute("SELECT addr,ver FR
    return {"ok": 1, "on": bool(TON_WALLET), "wallet": friendly_addr(raw_addr(TON
            "fee": TON_FEE, "mindep": TON_MIN_DEP, "minwd": TON_MIN_WD, "minlot":
            "wd": [dict(r) for r in c.execute("SELECT id,amount,status FROM ton_w

_TCK = {}
@api("/api/ton/check")
def ton_check(b):   # «я оплатил»: проверить поступления сейчас
    uid, _ = auth(b.get("initData")); t = time.time()
    if t - _TCK.get(uid, 0) < 5: raise HTTPException(429, "Подождите несколько се
    _TCK[uid] = t
    try: ton_poll(True)
    except Exception as e: log.warning("ton_poll: %s", e); raise HTTPException(50
    return {"ok": 1, "ton": ton_of(db(), uid)}

@api("/api/ton/tostars")
def ton_tostars(b):
    uid, _ = auth(b.get("initData")); c = db()
```

```
    try: n = int(b.get("s", 0))
    except (TypeError, ValueError): n = 0
    if n < 10 or n > 10000: raise HTTPException(400, "От 10 до 10 000 ⭐")
    cost = -(-n * NANO // TON_STAR_RATE)
    tspend(c, uid, cost, "tostars"); wadd(c, uid, n, "ton"); c.commit()
    return {"ok": 1, "ton": ton_of(c, uid), "stars": wallet_of(c, uid)}

@api("/api/ton/wd")
def ton_wd(b):
    uid, _ = auth(b.get("initData")); c = db()
    try: n = int(round(float(b.get("ton", 0)) * NANO))
    except (TypeError, ValueError): n = 0
    if n < TON_MIN_WD: raise HTTPException(400, "Минимум для вывода %.1f TON" % (
    w = c.execute("SELECT addr,ver FROM wallets WHERE uid=?", (uid,)).fetchone()
    if not w or not w["ver"]: raise HTTPException(400, "Вывод доступен на кошелёк
    if c.execute("SELECT COUNT(*) FROM ton_wd WHERE uid=? AND status='pending'",
    tspend(c, uid, n, "withdraw")
    cur = c.execute("INSERT INTO ton_wd(uid,amount,addr,ts) VALUES(?,?,?,?)", (ui
    for a in ADMIN_IDS: notify(a, "💸 Заявка на вывод #%d: %.3f TON, игрок %s\nКс
    return {"ok": 1, "ton": ton_of(c, uid), "id": cur.lastrowid}

def _tl_row(r): return {"id": r["id"], "plate": json.loads(r["plate"]), "price":
@api("/api/tl/list")
def tl_list(b):
    uid, _ = auth(b.get("initData")); c = db()
    rows = c.execute("SELECT l.*,u.name sn FROM ton_lot l LEFT JOIN users u ON u.
    return {"ok": 1, "list": [_tl_row(r) for r in rows], "ton": ton_of(c, uid)}

@api("/api/tl/create")
def tl_create(b):
    uid, _ = auth(b.get("initData")); c = db(); p = clean_plate(b.get("plate") or
    if tier_py(p) < 4: raise HTTPException(400, "За TON продаются только Мифик и
    try: price = int(round(float(b.get("price", 0)) * NANO))
    except (TypeError, ValueError): price = 0
    if price < TON_MIN_LOT or price > TON_MAX_LOT: raise HTTPException(400, "Цена
    if c.execute("SELECT COUNT(*) FROM ton_lot WHERE seller=? AND status='open'",
    if c.execute("UPDATE uniq SET owner=-3 WHERE key=? AND owner=?", (ukey(p), ui
        c.rollback(); raise HTTPException(400, "Этот единственный номер вам не пр
    cur = c.execute("INSERT INTO ton_lot(seller,plate,key,price,ts) VALUES(?,?,?,
    return {"ok": 1, "id": cur.lastrowid}

@api("/api/tl/buy")
def tl_buy(b):
    uid, _ = auth(b.get("initData")); c = db()
    try: lid = int(b.get("id", 0)); exp = int(b.get("price", 0))
    except (TypeError, ValueError): raise HTTPException(400, "Неверный лот")
    r = c.execute("SELECT * FROM ton_lot WHERE id=? AND status='open'", (lid,)).f
```

```
    if not r: raise HTTPException(400, "Лот уже продан или снят")
    if r["seller"] == uid: raise HTTPException(400, "Нельзя купить свой лот")
    if exp != r["price"]: raise HTTPException(400, "Цена изменилась, обновите спи
    tspend(c, uid, r["price"], "buy")
    if c.execute("UPDATE ton_lot SET status='sold',buyer=? WHERE id=? AND status=
        c.rollback(); raise HTTPException(400, "Лот уже продан или снят")
    fee = ton_fee(r["price"]); tadd(c, r["seller"], r["price"] - fee, "sale"); gi
    notify(r["seller"], "💎 Ваш номер продан за %.2f TON (комиссия %d%%). На бала
    return {"ok": 1, "ton": ton_of(c, uid)}

@api("/api/tl/cancel")
def tl_cancel(b):
    uid, _ = auth(b.get("initData")); c = db()
    try: lid = int(b.get("id", 0))
    except (TypeError, ValueError): raise HTTPException(400, "Неверный лот")
    r = c.execute("SELECT * FROM ton_lot WHERE id=? AND seller=? AND status='open
    if not r or c.execute("UPDATE ton_lot SET status='cancelled' WHERE id=? AND s
    gift(c, uid, {"t": "plate", "v": json.loads(r["plate"])}); c.commit(); return

@api("/api/nft/return")
def nft_return(b):
    """Вернуть NFT в игру: игрок отправляет NFT на кошелёк игры со своего (подтве
    uid, _ = auth(b.get("initData")); c = db()
    try: nid = int(b.get("id", 0))
    except (TypeError, ValueError): raise HTTPException(400, "Неверный NFT")
    r = c.execute("SELECT * FROM nft WHERE id=? AND uid=? AND status='minted'", (
    if not r: raise HTTPException(400, "NFT не найден")
    if not r["item"]: raise HTTPException(400, "Админ ещё не указал адрес этого N
    if not TON_WALLET: raise HTTPException(400, "Возврат NFT пока не настроен")
    w = c.execute("SELECT addr,ver FROM wallets WHERE uid=?", (uid,)).fetchone()
    if not w or not w["ver"]: raise HTTPException(400, "Сначала подтвердите кошел
    mine, game = raw_addr(w["addr"]), raw_addr(TON_WALLET)
    try:
        it = (ton_get("/nft/items", {"address": r["item"], "limit": 1}).get("nft_
        tr = ton_get("/nft/transfers", {"item_address": r["item"], "direction": "
    except Exception as e:
        log.warning("nft_return: %s", e); raise HTTPException(502, "Не удалось св
    if not it or raw_addr(it.get("owner_address")) != game: raise HTTPException(4
    if not any(raw_addr(t.get("new_owner")) == game and raw_addr(t.get("old_owner
        raise HTTPException(400, "NFT пришёл с другого кошелька, не с вашего. Обр
    p = json.loads(r["plate"])
    if c.execute("UPDATE nft SET status='returned',key=key||'#r'||id WHERE id=? A
    c.execute("UPDATE uniq SET owner=? WHERE key=? AND owner=-2", (uid, ukey(p)))
    return {"ok": 1}

# ---------------------------------------------------------------- рассылки от бо
TZ_OFF = 5 * 3600   # Ташкент
```

```
def push_users(c, where, args, text, btn="🎰 Открыть игру", limit=5000):
    for r in c.execute("SELECT id FROM users WHERE COALESCE(mute,0)=0 AND " + whe

def _meta_get(c, k):
    r = c.execute("SELECT v FROM meta WHERE k=?", (k,)).fetchone(); return r[0] i
def _meta_set(c, k, v): c.execute("INSERT OR REPLACE INTO meta(k,v) VALUES(?,?)",

DAILY_TXT = ["🎁 Ежедневный подарок уже ждёт тебя! Серия растёт — награды больше.
             "🔥 Не прерывай серию! Сегодняшний подарок ещё не забран.",
             "🎰 Бесплатная крутка и подарок дня на месте. Вдруг сегодня выпадет
             "💎 Мифик и Секретные номера — это NFT на TON. Зайди за подарком, вд
             "🚗 Гараж скучает! Забери подарок дня и сделай крутку.",
             "👀 Пока тебя не было, игроки выбивают редкие номера. Твоя очередь!"
             "🏁 Бросай вызов в гонках — ставки уже ждут. А подарок дня ещё не за
             "💰 Подарок дня — это сумы, крутки и бусты удачи. Заходи, пока не сг
             "⚡ Один заход — и подарок твой. Всего 5 секунд!",
             "🍀 Сегодня удача на твоей стороне? Проверь: подарок дня + бесплатна
             "🏆 Идёт сезон: гонки, задания и Battle Pass ждут. Заходи, забери пс
             "🔥 Серия входов даёт всё больше наград. Не теряй её, зайди сегодня!
DAILY_BTN = ["🎁 Забрать подарок", "🎰 Крутить номер", "🔥 Продолжить серию", "🏁
def daily_reminders(c):
    """Раз в день каждому игроку, который сегодня ещё не заходил: напоминание о п
    lt = time.gmtime(time.time() + TZ_OFF); day = int((time.time() + TZ_OFF) // 8
    if hour < 10 or hour > 22: return 0
    t0 = day * 86400 - TZ_OFF; n = 0
    rows = c.execute("SELECT id FROM users WHERE COALESCE(mute,0)=0 AND COALESCE(
                     (day, t0 - 30 * 86400, t0, day * 17, hour)).fetchall()
    for r in rows:
        c.execute("UPDATE users SET rem=? WHERE id=?", (day, r[0])); notify(r[0],
    c.commit(); return n

WEEKEND_EVENT = {"luck": 1.5, "sell": 1.0, "xp": 2.0, "upg": 1.0}
def weekend_event(c):
    """Суббота и воскресенье: автоматический ивент (удача ×1.5, опыт ×2, апгрейд
    lt = time.gmtime(time.time() + TZ_OFF)
    if lt.tm_wday not in (5, 6): return
    now = int(time.time())
    key = "wk%d" % ((now + TZ_OFF) // 86400 - lt.tm_wday)   # номер недели
    if _meta_get(c, key) or c.execute("SELECT 1 FROM events WHERE until>?", (now,
    until = ((now + TZ_OFF) // 86400 + (7 - lt.tm_wday)) * 86400 - TZ_OFF   # пон
    c.execute("DELETE FROM events"); c.execute("INSERT INTO events(name,luck,sell
    c.commit(); _meta_set(c, key, now); _EVC[0] = 0
    push_users(c, "last_seen>?", (now - 14 * 86400,), "🎉 Выходные в UZ Номерок!

def drop_alert(c, uid, p):
    """Кто-то выбил Мифик/Секретный: сообщить остальным (не чаще раза в 30 минут
    now = int(time.time()); last = int(_meta_get(c, "dropnote") or 0)
```

```
    if now - last < 1800: return
    _meta_set(c, "dropnote", now); u = c.execute("SELECT name FROM users WHERE id
    t = tier_py(p); txt = "🔥 Игрок %s (ID %d) выбил %s номер %s! Заходи и ты — т
    push_users(c, "last_seen>? AND id!=?", (now - 3 * 86400, uid), txt)

@api("/api/send")
def send_plate(b):
    """Передать номер другому игроку по ID (Мифик/Секретные: принадлежность прове
    uid, _ = auth(b.get("initData")); c = db(); p = clean_plate(b.get("p") or {})
    try: to = int(b.get("to", 0))
    except (TypeError, ValueError): raise HTTPException(400, "Неверный ID")
    if to == uid or not c.execute("SELECT 1 FROM users WHERE id=?", (to,)).fetcho
    day = int(time.time() // 86400)
    if c.execute("SELECT COUNT(*) FROM tlog WHERE uid=? AND why='send' AND ts>?",
    if tier_py(p) >= 4 and c.execute("UPDATE uniq SET owner=? WHERE key=? AND own
        c.rollback(); raise HTTPException(400, "Этот единственный номер вам не пр
    c.execute("INSERT INTO tlog(uid,d,why,ts) VALUES(?,0,'send',?)", (uid, int(ti
    gift(c, to, {"t": "plate", "v": p}); c.commit()
    me = c.execute("SELECT name FROM users WHERE id=?", (uid,)).fetchone()
    notify(to, "🎁 Игрок %s (ID %d) передал вам номер %s! Он уже в гараже." % (((
    return {"ok": 1}

os.makedirs(os.path.join(HERE, "cars"), exist_ok=True)

class CarFiles(StaticFiles):   # фото машин кэшируются в телефоне на неделю
    def file_response(self, *a, **k):
        r = super().file_response(*a, **k); r.headers["Cache-Control"] = "public,

app.mount("/cars", CarFiles(directory=os.path.join(HERE, "cars")), name="cars")
_CI = [0, []]

@api("/api/carimgs", method="get")
def carimgs(b):   # какие фото машин реально лежат в папке cars (игра не просит н
    t = time.time()
    if t - _CI[0] > 60:
        try: _CI[1] = sorted(f[:-4] for f in os.listdir(os.path.join(HERE, "cars"
        except OSError: _CI[1] = []
        _CI[0] = t
    return {"list": _CI[1]}

# ------------------------------------------------------------------ подписка на ка
_GATE = {}   # (канал, игрок) -> (до какого времени верить, подписан, когда прове

def member_ok(chat, uid, force=False):
    key = (chat, uid); t = time.time(); e = _GATE.get(key)
    if e and e[0] > t and not (force and not e[1] and t - e[2] > 3): return e[1]
    try:
```

```
        res = tg_request("getChatMember?chat_id=%s&user_id=%d" % (urllib.parse.qu
        ok = res.get("result", {}).get("status") in ("member", "administrator", "
    except Exception as ex:
        log.warning("getChatMember не ответил: %s", type(ex).__name__)
        ok, ttl = True, 60   # не удалось проверить (бот не админ?) - не блокируе
    if len(_GATE) > 50000: _GATE.clear()
    _GATE[key] = (t + ttl, ok, t); return ok

@api("/api/gate", pool=TGPOOL)
def gate(b):
    uid, _ = auth(b.get("initData"))
    if not REQUIRED_CHANNEL: return {"ok": True}
    return {"ok": member_ok(REQUIRED_CHANNEL, uid, bool(b.get("force"))), "url":

@api("/api/sub", pool=TGPOOL)
def sub(b):
    uid, _ = auth(b.get("initData")); cid = b.get("id"); c = db()
    ch = CHANNELS.get(cid)
    if not ch: raise HTTPException(400, "Канал не найден")
    if c.execute("SELECT 1 FROM claims WHERE uid=? AND id=?", (uid, cid)).fetchon
    try: res = tg_request("getChatMember?chat_id=%s&user_id=%d" % (urllib.parse.q
    except Exception: raise HTTPException(400, "Не удалось проверить подписку")
    if res.get("result", {}).get("status") not in ("member", "administrator", "cr
        raise HTTPException(400, "Сначала подпишитесь на канал")
    try: c.execute("INSERT INTO claims VALUES(?,?)", (uid, cid)); c.commit()
    except sqlite3.IntegrityError: raise HTTPException(400, "Уже получено")
    return {"ok": 1}

@api("/api/event")
def event(b):
    auth(b.get("initData")); return event_of(db())

REGS = ["01", "10", "20", "25", "30", "40", "50", "60", "70", "75", "80", "85", "
LETTERS = set("ABCDEFGHJKLMNPRSTUVWXYZ")
BLOCK = {"PAA", "PSS", "PPP", "APC", "TSS", "TST", "ZDF", "DXA", "FVV", "VSB", "V
         "TTA", "TTB", "TTC", "TTD", "TTE", "TTF", "TTG", "UZB", "AAA", "SBA", "A
STAR_ITEMS = {  # item: (звёзды, название, описание)
    "b50": (50, "Удача x2 на 2 часа", "Повышенный шанс на хорошие номера на 2 час
    "b100": (100, "Набор бустов на 2 часа", "Удача x2, опыт x2 и бонус продажи +2
    "p500": (500, "Любой номер на выбор", "Номер, который вы выберете сами"),
    "noads": (99, "Убрать рекламу", "Реклама в игре отключается навсегда"),
}

def has_noads(c, uid): return c.execute("SELECT 1 FROM payments WHERE uid=? AND i

# ---------------------------------------------------------------- звёзды: пополн
STAR_RATE = int(os.environ.get("STAR_RATE", "1000000"))   # сколько сум даём за 1
```

```
STAR_MIN, STAR_MAX = 50, 5000
WALLET_MIN, WALLET_MAX = 10, 10000                          # пополнение звёздного
SA_FEE, SA_MIN = 5, 10                                     # комиссия звёздного ау
STAR_BONUS = [(1000, 25), (250, 10)]                        # от 250 ⭐ +10%, от 1
SPIN_PRICE = 15                                             # одна звёздная крутка
SPIN_PACKS = (1, 5, 10)
SPIN_MYTHIC = 0.10                                          # шанс Мифика в крутке
SPIN_PAA = 0.03                                             # шанс номера PAA в кр
SPIN_SAME = 0.05                                            # шанс одинаковых цифр
SPIN_SECRET = 0.10                                          # шанс Секретного номе

def star_sum(stars):
    pct = next((p for lim, p in STAR_BONUS if stars >= lim), 0)
    return int(stars * STAR_RATE * (100 + pct) / 100)

@api("/api/stars")
def stars_info(b):
    auth(b.get("initData"))
    return {"rate": STAR_RATE, "min": STAR_MIN, "max": STAR_MAX, "bonus": [[lim,
            "packs": list(SPIN_PACKS), "secret": 100 - int(SPIN_MYTHIC * 100) - i

_PLR = ["01", "10", "20", "25", "30", "40", "50", "60", "70", "75", "80", "85", "
_ULTRA = [{"k": 1, "r": "01", "d": "879", "z": "MEA"}, {"k": 1, "r": "01", "d": "
          {"k": 0, "r": "01", "a": "Z", "d": "777", "z": "ZZ"}, {"k": 0, "r": "01

def _claim(c, uid, p):   # атомарно занять единственный номер: получится только у
    return c.execute("INSERT OR IGNORE INTO uniq(key,owner,ts) VALUES(?,?,?)", (u

def _pretty(d): return d[0] == d[1] == d[2] or d in ("123", "321", "456", "789",

SER_REG = {"SBA": ("25", "40"), "AAA": ("25", "40"), "UZB": ("25", "40", "01")}
PAA_REG = ("25", "40", "01")

def roll_unique(c, uid, reg=""):
    """Крутка за звёзды. Регион можно выбрать (reg), пусто - любой.
    PAA (3%) и Секретный (спецсерии SBA/AAA/UZB) - только в своих регионах: SBA/A
    Мифик (тройные цифры) - в любом регионе. Нет подходящего - Мифик; свободных н
    if reg not in _PLR: reg = ""
    if _RND.random() < SPIN_PAA and (not reg or reg in PAA_REG):
        for _ in range(400):
            p = {"k": 1, "r": reg or _RND.choice(PAA_REG), "d": "%03d" % _RND.ran
            if tier_py(p) >= 4 and _claim(c, uid, p): return p
    sers = [z for z in ("SBA", "AAA", "UZB") if not reg or reg in SER_REG[z]]
    want_sec = bool(sers) and _RND.random() >= SPIN_MYTHIC
    if want_sec:
        w = {"SBA": 40, "AAA": 40, "UZB": 20}
        for _ in range(800):
```

```
            z = _RND.choices(sers, [w[x] for x in sers])[0]
            r = reg or _RND.choice(SER_REG[z])
            if _RND.random() < SPIN_SAME: d = str(_RND.randint(1, 9)) * 3
            else:
                d = "%03d" % _RND.randint(1, 999)
                if _pretty(d): continue
            p = {"k": 1, "r": r, "d": d, "z": z}
            if tier_py(p) == 5 and _claim(c, uid, p): return p
    for _ in range(800):   # Мифик (или запасной вариант)
        x = str(_RND.randint(1, 9)); b = _RND.choice(sorted(LETTERS))
        if reg not in ("", "01") or _RND.random() < 0.5: p = {"k": 1, "r": reg or
        else: p = {"k": 0, "r": "01", "a": _RND.choice(sorted(LETTERS)), "d": x *
        if tier_py(p) == 4 and _claim(c, uid, p): return p
    return None

def apply_payment(c, d, total, charge):
    """Выдать оплаченное. Возвращает текст для игрока или None, если платёж уже о
    uid = int(d["u"]); item = d.get("i")
    try: c.execute("INSERT INTO payments VALUES(?,?,?,?,?)", (charge, uid, item,
    except sqlite3.IntegrityError: return None
    msg = "⭐ Оплата получена! Откройте игру: подарок уже там."
    if item == "b50": gift(c, uid, {"t": "boost", "k": "bl", "v": 120})
    elif item == "b100":
        for k in ("bl", "bx", "bs"): gift(c, uid, {"t": "boost", "k": k, "v": 120
    elif item == "noads": gift(c, uid, {"t": "noads"})
    elif item == "wallet":
        st = int(d.get("s", 0))
        if st != total or not (WALLET_MIN <= st <= WALLET_MAX): log.error("платёж
        wadd(c, uid, st, "topup"); msg = "⭐ Звёздный баланс пополнен на %d ⭐. Т
    elif item == "p500": gift(c, uid, {"t": "plate", "v": d["p"]})
    elif item == "bal":
        st = int(d.get("s", 0))
        if st != total or not (STAR_MIN <= st <= STAR_MAX): log.error("платёж bal
        gift(c, uid, {"t": "bal", "v": star_sum(st)}); msg = "⭐ Оплата получена!
    elif item == "spin":
        n = int(d.get("n", 0))
        if n not in SPIN_PACKS or total < n * SPIN_PRICE: log.error("платёж spin:
        got = []
        for _ in range(n):
            p = roll_unique(c, uid, str(d.get("g") or ""))
            if p: gift(c, uid, {"t": "plate", "v": p, "src": "spin"}); got.append
            else: gift(c, uid, {"t": "bal", "v": SPIN_PRICE * STAR_RATE})   # еди
        msg = "🎰 Звёздная крутка! Ваши единственные номера:\n" + "\n".join("🔢 '
    c.commit()
    return msg

def check_plate(p):  # проверка номера, который игрок хочет купить за звёзды
```

```
    try:
        k = int(p["k"]); r = str(p["r"]); d = str(p["d"]); z = str(p["z"]).upper(
    except Exception: raise HTTPException(400, "Неверный номер")
    bad = r not in REGS or not (d.isdigit() and len(d) == 3) or d == "000" or not
    if k == 1: bad = bad or len(z) != 3
    else: bad = bad or len(z) != 2 or len(a) != 1 or a not in LETTERS
    if not bad and tier_py({"k": k, "r": r, "d": d, "z": z, "a": a}) >= 4: bad =
    if bad or (a + z) in BLOCK or z in BLOCK or (k == 0 and r == "01" and a == "R
        raise HTTPException(400, "Этот номер купить нельзя")
    return {"k": k, "r": r, "d": d, "z": z, **({"a": a} if k == 0 else {})}

@api("/api/invoice", pool=TGPOOL)
def invoice(b):
    uid, _ = auth(b.get("initData")); item = b.get("item")
    if item == "bal":
        try: stars = int(b.get("s", 0))
        except (TypeError, ValueError): raise HTTPException(400, "Неверная сумма"
        if not (STAR_MIN <= stars <= STAR_MAX): raise HTTPException(400, "Пополне
        amount, title, desc = stars, "Пополнение баланса", "%s сум на игровой бал
    elif item == "wallet":
        try: stars = int(b.get("s", 0))
        except (TypeError, ValueError): raise HTTPException(400, "Неверная сумма"
        if not (WALLET_MIN <= stars <= WALLET_MAX): raise HTTPException(400, "Поп
        amount, title, desc = stars, "Звёздный баланс", "Пополнение звёздного бал
    elif item == "spin":
        try: n = int(b.get("n", 0))
        except (TypeError, ValueError): raise HTTPException(400, "Неверное количе
        if n not in SPIN_PACKS: raise HTTPException(400, "Неверное количество")
        reg = str(b.get("reg") or "")
        if reg and reg not in _PLR: raise HTTPException(400, "Неверный регион")
        amount, title, desc = n * SPIN_PRICE, "Звёздная крутка x%d" % n, "Выпадае
        if reg: payload["g"] = reg
    elif item not in STAR_ITEMS: raise HTTPException(400, "Товар не найден")
    else:
        if item == "noads" and has_noads(db(), uid): raise HTTPException(400, "Ре
        amount, title, desc = STAR_ITEMS[item]; payload = {"u": uid, "i": item}
    if item == "p500": payload["p"] = check_plate(b.get("plate") or {})
    pl = json.dumps(payload, separators=(",", ":"))
    try: res = tg_request("createInvoiceLink", {"title": title, "description": de
                                                "prices": [{"label": title, "amou
    except urllib.error.HTTPError as e:   # Telegram отказал: покажем точную прич
        try: desc = json.load(e).get("description", "")
        except Exception: desc = ""
        log.error("createInvoiceLink %s отклонён: %s %s", item, e.code, desc)
        raise HTTPException(400, "Telegram: %s" % (desc[:90] or "счёт не создан")
    except Exception as e:
        log.error("createInvoiceLink %s ошибка: %r", item, e)
```

```
        raise HTTPException(400, "Не удалось создать счёт, попробуйте ещё раз")
    if not res.get("ok"):
        log.error("createInvoiceLink %s: %s", item, res)
        raise HTTPException(400, "Telegram: %s" % str(res.get("description", "счё
    return {"url": res["result"]}

# ---------------------------------------------------------------- аукцион между
def settle(c):  # завершить аукционы, у которых вышло время
    now = int(time.time()); notes = []
    for a in c.execute("SELECT * FROM auctions WHERE status='open' AND end<=?", (
        if c.execute("UPDATE auctions SET status='done' WHERE id=? AND status='op
        item = json.loads(a["item"]); nm = item_txt(item)
        if a["topuid"]:
            gift(c, a["topuid"], item)
            notes.append((a["topuid"], "🏆 Вы выиграли аукцион!\n%s\nВаша ставка:
            if a["seller"]:
                got = int(a["top"] * 0.95); gift(c, a["seller"], {"t": "bal", "v"
                notes.append((a["seller"], "✅ Ваш лот продан!\n%s\nЦена: %s сум.
        elif a["seller"]:
            gift(c, a["seller"], item)   # ставок не было - предмет возвращается
            notes.append((a["seller"], "⌛ Торги закончились без ставок.\n%s\nПре
    c.commit()
    for n in notes: notify(*n)

def next_price(a): return a["start"] if not a["topuid"] else math.ceil(a["top"] *

_AUC = [0, []]
def auc_snapshot(c):   # список лотов: один раз в секунду на всех игроков сразу
    t = time.time()
    if t - _AUC[0] < 1.0: return _AUC[1]
    if c.execute("SELECT 1 FROM auctions WHERE status='open' AND end<=? LIMIT 1",
    rows = c.execute("SELECT * FROM auctions WHERE status='open' ORDER BY end LIM
    _AUC[1] = [{"id": a["id"], "seller": a["seller"], "item": json.loads(a["item"
                "top": a["top"], "topuid": a["topuid"], "end": a["end"], "next":
    _AUC[0] = t; return _AUC[1]

@api("/api/auc/create")
def auc_create(b):
    uid, _ = auth(b.get("initData")); c = db(); item = b.get("item") or {}
    try: start = int(b.get("start", 0)); mins = int(b.get("mins", 0))
    except (TypeError, ValueError): raise HTTPException(400, "Неверные данные лот
    if item.get("t") not in ("plate", "car") or not isinstance(item.get("v"), dic
        raise HTTPException(400, "Неверные данные лота")
    need_own(c, uid, item["v"])
    if c.execute("SELECT COUNT(*) FROM auctions WHERE seller=? AND status='open'"
        raise HTTPException(400, "Не больше 10 лотов одновременно")
    c.execute("INSERT INTO auctions(seller,item,start,end) VALUES(?,?,?,?)", (uid
```

```
    _AUC[0] = 0
    return {"ok": 1}

@api("/api/auc/list")
def auc_list(b):   # список лотов + подарки игрока одним запросом
    uid, _ = auth(b.get("initData")); c = db()
    return {"now": int(time.time()), "list": auc_snapshot(c), "gifts": gifts_of(c

@api("/api/auc/bid")
def auc_bid(b):
    uid, _ = auth(b.get("initData")); c = db(); settle(c); now = int(time.time())
    try: aid = int(b["id"])
    except (KeyError, TypeError, ValueError): raise HTTPException(400, "Неверные
    a = c.execute("SELECT * FROM auctions WHERE id=? AND status='open' AND end>?"
    if not a: raise HTTPException(400, "Торги уже закончились")
    if a["seller"] == uid: raise HTTPException(400, "Нельзя ставить на свой лот")
    if a["topuid"] == uid: raise HTTPException(400, "Вы уже лидер")
    price = next_price(a)
    if price != int(b.get("exp", -1)): raise HTTPException(400, "Цена изменилась,
    ok = c.execute("UPDATE auctions SET top=?,topuid=?,end=MAX(end,?) WHERE id=?
                   (price, uid, now + 15, a["id"], now, a["top"], a["topuid"] or
    if ok != 1: raise HTTPException(400, "Вашу ставку перебили, попробуйте ещё ра
    if a["topuid"]: gift(c, a["topuid"], {"t": "bal", "v": a["top"]})   # возврат
    c.commit(); _AUC[0] = 0
    if a["topuid"]:
        notify(a["topuid"], random.choice(["🔨 Вашу ставку перебили!\n%s\nНовая с
            item_txt(json.loads(a["item"])), fnum(price), fnum(a["top"]), left_tx
    return {"ok": 1, "price": price}

@api("/api/auc/cancel")
def auc_cancel(b):   # продавец снимает свой лот, если ставок ещё нет
    uid, _ = auth(b.get("initData")); c = db()
    try: aid = int(b["id"])
    except (KeyError, TypeError, ValueError): raise HTTPException(400, "Неверные
    a = c.execute("SELECT * FROM auctions WHERE id=? AND status='open'", (aid,)).
    if not a or a["seller"] != uid: raise HTTPException(400, "Лот недоступен")
    if c.execute("UPDATE auctions SET status='cancelled' WHERE id=? AND status='o
        raise HTTPException(400, "На лот уже сделали ставку, снять нельзя")
    gift(c, uid, json.loads(a["item"])); c.commit(); _AUC[0] = 0
    return {"ok": 1}

# ---------------------------------------------------------------- покупки за звё
def _same_pl(x, p):   # тот же номер с тем же множителем
    if not isinstance(x, dict): return False
    try: q = clean_plate(x)
    except HTTPException: return False
    return all(q.get(k) == p.get(k) for k in ("k", "r", "d", "z", "a")) and (q.ge
```

```
def _upg_apply(st, p, win, newm):   # результат апгрейда применяется на сервере:
    col = st.get("col")
    if not isinstance(col, list): return False
    for i, x in enumerate(col):
        if _same_pl(x, p):
            if win: x["m"] = newm
            else: col.pop(i)
            return True
    return False

@api("/api/upg")
def upg(b):
    """Апгрейд-колесо: выбираем множитель x2/x5/x10, шанс = 94%/множитель. Выигры
    (со страховкой за ⭐ номер остаётся). Бросок делает сервер."""
    uid, _ = auth(b.get("initData")); c = db(); p = clean_plate(b.get("p") or {})
    try: x = int(b.get("x", 0))
    except (TypeError, ValueError): x = 0
    if x not in UPG_X: raise HTTPException(400, "Неверный множитель")
    if (p.get("m") or 1) * x > UPG_MAXM: raise HTTPException(400, "Максимальный м
    need_own(c, uid, p)
    ur = c.execute("SELECT state FROM users WHERE id=?", (uid,)).fetchone()
    try: ust = json.loads(ur["state"]) if ur and ur["state"] else {}
    except Exception: ust = {}
    if not any(_same_pl(x, p) for x in (ust.get("col") or [])): raise HTTPExcepti
    ins = bool(b.get("ins"))
    if ins: wspend(c, uid, UPG_INS, "upg-ins")
    rtp = float(event_of(c).get("upg") or UPG_RTP); ch = min(.95, round(rtp / x *
    win = random.SystemRandom().random() < ch
    if not win and not ins and tier_py(p) >= 4:
        c.execute("UPDATE uniq SET owner=0 WHERE key=? AND owner=?", (ukey(p), ui
    newm = (p.get("m") or 1) * x
    if win or not ins:   # итог сразу записываем в сохранение игрока и держим 2 м
        _upg_apply(ust, p, win, newm)
        c.execute("UPDATE users SET state=? WHERE id=?", (json.dumps(ust, separat
        _pend_add(c, uid, p, 1 if win else 0, newm, _count_pl(ust, p))
    c.commit()
    return {"ok": 1, "win": win, "ch": ch, "ins": ins, "m": newm if win else (p.g


# ---------------------------------------------------------------- сезон, скидка
SEASON_DEF = os.environ.get("SEASON_NAME", "Beta 2")
BP_PRICE = {1: 100, 2: 250}            # Премиум 100 ⭐, Премиум+ 250 ⭐ (если уж
DEAL_PCT = 40
DEAL_ITEMS = ("b50", "b100", "spin1", "spin5", "spin10")

def season_of(c):
```

```
    n = c.execute("SELECT v FROM meta WHERE k='season_name'").fetchone(); n = n[0
    e = c.execute("SELECT v FROM meta WHERE k='season_end'").fetchone()
    if not e:
        end = int(time.time()) + 14 * 86400
        c.execute("INSERT OR IGNORE INTO meta(k,v) VALUES('season_end',?)", (str(
    else: end = int(e[0])
    return n, end

def bp_tier(c, uid):
    n, _ = season_of(c)
    r = c.execute("SELECT tier FROM bp WHERE uid=? AND sk=?", (uid, n)).fetchone(
    return int(r[0]) if r else 0

def deal_day(): return (int(time.time()) + 5 * 3600) // 86400          # день по
def deal_item(): return DEAL_ITEMS[deal_day() % len(DEAL_ITEMS)]
def base_price(item):
    if item.startswith("spin"): return int(item[4:]) * SPIN_PRICE
    return STAR_ITEMS[item][0]
def deal_price(item): return max(1, -(-base_price(item) * (100 - DEAL_PCT) // 100

def season_info(c, uid):
    n, end = season_of(c); it = deal_item()
    used = c.execute("SELECT 1 FROM deal_used WHERE uid=? AND day=?", (uid, deal_
    return {"n": n, "end": end, "bp": bp_tier(c, uid), "bpp": BP_PRICE, "deal": {
            "ads": 0 if ads_is_off(c) else 1, "ai": ads_cfg(c)}

def ads_cfg(c):
    r = c.execute("SELECT v FROM meta WHERE k='ads_cfg'").fetchone()
    try: i, f = [int(x) for x in r[0].split(",")]
    except Exception: i, f = 90, 4
    return [max(20, min(3600, i)), max(1, min(100, f))]

def ads_is_off(c):
    r = c.execute("SELECT v FROM meta WHERE k='ads_off'").fetchone(); return bool

# ------------------------------------------------------------- итоги сезона и
SEASON_BAL = int(os.environ.get("SEASON_BAL", 50_000_000))   # сколько сумов у ка
SEASON_MIN_WINS = 3     # чтобы попасть в топ по победам в гонках
SEASON_MIN_LV = 3       # чтобы попасть в топ по уровню
_M = 1_000_000
# награды: m - сумы, s - звёзды, cs - косметика (ключ из COSM), bp - Battle Pass
REW_WINS = [(1, 1, {"m": 150 * _M, "s": 200, "cs": ["gold", "chrome", "pearl", "c
            (2, 2, {"m": 100 * _M, "s": 120, "cs": ["chrome", "pearl", "fl_white"
            (3, 3, {"m": 70 * _M, "s": 80, "cs": ["pearl", "fl_blue"]}), (4, 10,
REW_LVL = [(1, 1, {"m": 100 * _M, "s": 100, "cs": ["fl_gold", "fl_pink", "fl_gree
           (3, 3, {"m": 50 * _M, "s": 40, "cs": "fl_green"}), (4, 10, {"m": 20 *
REW_PART = [(30, {"m": 40 * _M, "fp": 10, "fc": 2, "bl": 180, "cs": "fl_blue"}),
```

```
def _meta(c, k, d=0):
    r = c.execute("SELECT v FROM meta WHERE k=?", (k,)).fetchone()
    try: return int(r[0]) if r else d
    except (TypeError, ValueError): return d

def _rstat_upsert(c, uid, sk, win):
    c.execute("INSERT INTO rstat(uid,sk,played,wins) VALUES(?,?,1,?) ON CONFLICT(

def _rstat_backfill(c):   # один раз за сезон: заполняем статистику уже сыгранным
    n = season_of(c)[0]; r = c.execute("SELECT v FROM meta WHERE k='rstat_bf'").f
    if r and r[0] == n: return False
    since = _meta(c, "season_start", 0)
    for x in c.execute("SELECT a,b,winner FROM races WHERE status='done' AND ts>=
        for u in (x["a"], x["b"]):
            if u: _rstat_upsert(c, u, n, x["winner"] == u)
    c.execute("INSERT OR REPLACE INTO meta(k,v) VALUES('rstat_bf',?)", (n,)); ret

def rstat_add(c, uid, win):
    try:
        if _rstat_backfill(c): return   # текущий заезд уже в таблице races и пос
        _rstat_upsert(c, uid, season_of(c)[0], win)
    except Exception: log.exception("rstat")

def _merge(a, b):
    for k, v in b.items():
        if k == "cs": a.setdefault("cs", []).extend(v if isinstance(v, list) else
        elif k == "bp": a["bp"] = max(a.get("bp", 0), v)
        else: a[k] = a.get(k, 0) + v

def season_plan(c):
    """Кто что получает за текущий сезон. Ничего не меняет в базе (кроме разового
    n, _e = season_of(c); _rstat_backfill(c); since = _meta(c, "season_start", 0)
    rs = {r["uid"]: (r["played"], r["wins"]) for r in c.execute("SELECT uid,playe
    P = {}
    for u in c.execute("SELECT id,name,last_seen,state FROM users WHERE state IS
        try: st = json.loads(u["state"]) or {}
        except Exception: st = {}
        p, w = rs.get(u["id"], (0, 0))
        P[u["id"]] = {"name": str(u["name"] or "Игрок")[:18], "p": p, "w": w, "lv
                      "cr": len(st.get("cars") or []), "sp": int(st.get("sp") or
    wl = sorted([i for i, x in P.items() if not x["hid"] and x["w"] >= SEASON_MIN
    ll = sorted([i for i, x in P.items() if not x["hid"] and x["act"] and x["lv"]
    for lst, tbl, key in ((wl, REW_WINS, "wr"), (ll, REW_LVL, "lr")):
        for pos, i in enumerate(lst, 1):
            for lo, hi, r in tbl:
                if lo <= pos <= hi: P[i][key] = pos; _merge(P[i]["rew"], r); brea
```

```
    for i, x in P.items():
        if x["hid"]: continue
        for need, r in REW_PART:
            if x["p"] >= need: _merge(x["rew"], r); break
    return n, P

def rew_text(r):
    o = []
    if r.get("m"): o.append("%s сум" % fnum(r["m"]))
    if r.get("s"): o.append("%d ⭐" % r["s"])
    if r.get("fp"): o.append("%d круток номера" % r["fp"])
    if r.get("fc"): o.append("%d круток машины" % r["fc"])
    if r.get("bl"): o.append("удача ×2 на %s" % ("%d ч" % (r["bl"] // 60) if r["b
    for k in r.get("cs", []): o.append("🎨 " + COSM_NAME.get(k, k))
    if r.get("bp"): o.append("Battle Pass %s на новый сезон" % ("Премиум+" if r["
    return o

COSM_NAME = {"chrome": "краска Хром", "gold": "краска Золото", "matte": "краска М
             "fl_blue": "синее пламя", "fl_green": "зелёное пламя", "fl_pink": "р

def season_apply(c, P, old, new, days):
    """Новый сезон: у всех сумы = SEASON_BAL (номера и машины остаются), награды
    now = int(time.time())
    for uid, x in P.items():
        r = x["rew"]
        gift(c, uid, {"t": "setbal", "v": SEASON_BAL})
        if r.get("m"): gift(c, uid, {"t": "bal", "v": r["m"]})
        if r.get("bl"): gift(c, uid, {"t": "boost", "k": "bl", "v": r["bl"]})
        if r.get("fp"): gift(c, uid, {"t": "fp", "v": r["fp"]})
        if r.get("fc"): gift(c, uid, {"t": "fc", "v": r["fc"]})
        if r.get("s"): wadd(c, uid, r["s"], "season-reward")
        for k in r.get("cs", []): c.execute("INSERT OR IGNORE INTO cosm VALUES(?,
        if r.get("bp"): c.execute("INSERT OR REPLACE INTO bp(uid,sk,tier) VALUES(
        if x["act"] and not x["hid"]:
            gift(c, uid, {"t": "recap", "v": {"n": old, "nx": new, "p": x["p"], "
    c.execute("INSERT OR REPLACE INTO meta(k,v) VALUES('season_name',?)", (new,))
    c.execute("INSERT OR REPLACE INTO meta(k,v) VALUES('season_start',?)", (str(n
    for uid, x in P.items():   # личные сообщения тем, кто в призах
        if x["hid"] or not (x["wr"] or x["lr"] or x["p"] >= 10): continue
        notify(uid, "🏁 Итоги сезона %s\n%s\nНаграды: %s\n\nНовый сезон %s началс
            old, "Гонок: %d, побед: %d%s%s" % (x["p"], x["w"], (", место по побед

@api("/api/bp/buy")
def bp_buy(b):
    uid, _ = auth(b.get("initData")); c = db()
    try: tier = int(b.get("tier", 0))
    except (TypeError, ValueError): raise HTTPException(400, "Неверный уровень")
```

```
    if tier not in BP_PRICE: raise HTTPException(400, "Неверный уровень")
    n, _e = season_of(c); cur = bp_tier(c, uid)
    if cur >= tier: raise HTTPException(400, "У вас уже есть этот Battle Pass")
    price = BP_PRICE[tier] - (BP_PRICE[cur] if cur else 0)
    wspend(c, uid, price, "bp:%s:%d" % (n, tier))
    c.execute("INSERT OR REPLACE INTO bp(uid,sk,tier) VALUES(?,?,?)", (uid, n, ti
    return {"ok": 1, "tier": tier, "stars": wallet_of(c, uid)}

@api("/api/wbuy")
def wbuy(b):
    """Купить что-то за ⭐ со звёздного баланса (пополнение баланса - отдельная с
    uid, _ = auth(b.get("initData")); c = db(); item = b.get("item"); d = {"u": u
    if item == "spin":
        try: n = int(b.get("n", 0))
        except (TypeError, ValueError): raise HTTPException(400, "Неверное количе
        if n not in SPIN_PACKS: raise HTTPException(400, "Неверное количество")
        reg = str(b.get("reg") or "")
        if reg and reg not in _PLR: raise HTTPException(400, "Неверный регион")
        price = n * SPIN_PRICE; d["n"] = n
        if reg: d["g"] = reg
    elif item == "p500": d["p"] = check_plate(b.get("plate") or {}); price = STAR
    elif item in ("b50", "b100"): price = STAR_ITEMS[item][0]
    elif item == "noads":
        if has_noads(c, uid): raise HTTPException(400, "Реклама уже отключена")
        price = STAR_ITEMS["noads"][0]
    elif item == "bal":
        try: st = int(b.get("s", 0))
        except (TypeError, ValueError): raise HTTPException(400, "Неверная сумма"
        if not (STAR_MIN <= st <= STAR_MAX): raise HTTPException(400, "Обмен от %
        price = st; d["s"] = st
    else: raise HTTPException(400, "Товар не найден")
    key = ("spin%d" % d["n"]) if item == "spin" else item; deal = False
    if key == deal_item() and not d.get("g") and item not in ("bal", "p500", "noa
        if c.execute("SELECT 1 FROM deal_used WHERE uid=? AND day=?", (uid, deal_
            price = deal_price(key); deal = True
    wspend(c, uid, price, "buy:" + item)
    if deal: c.execute("INSERT OR IGNORE INTO deal_used(uid,day) VALUES(?,?)", (u
    msg = apply_payment(c, d, price, "wallet:" + os.urandom(8).hex())
    if msg is None:
        wadd(c, uid, price, "refund:" + item)
        if deal: c.execute("DELETE FROM deal_used WHERE uid=? AND day=?", (uid, d
        c.commit(); raise HTTPException(400, "Не удалось выдать покупку, звёзды в
    c.commit()
    return {"ok": 1, "stars": wallet_of(c, uid), "msg": msg, "item": item}

# ------------------------------------------------------------------ аукцион за звё
def sa_next(a): return a["start"] if not a["topuid"] else a["top"] + max(1, math.
```

```
def sa_settle(c):
    now = int(time.time()); notes = []
    for a in c.execute("SELECT * FROM star_auc WHERE status='open' AND end<=?", (
        if c.execute("UPDATE star_auc SET status='done' WHERE id=? AND status='op
        p = json.loads(a["item"]); nm = plate_txt(p)
        if a["topuid"]:
            fee = a["top"] * SA_FEE // 100; got = a["top"] - fee
            gift(c, a["topuid"], {"t": "plate", "v": p}); wadd(c, a["seller"], go
            notes.append((a["topuid"], "🏆 Вы выиграли аукцион за ⭐!\n🔢 %s\nСт
            notes.append((a["seller"], "✅ Ваш номер продан за ⭐!\n🔢 %s\nЦена:
        else:
            gift(c, a["seller"], {"t": "plate", "v": p})
            notes.append((a["seller"], "⏳ Торги за ⭐ закончились без ставок.\n
    c.commit()
    for n in notes: notify(*n)

_SA = [0, []]
def sa_snapshot(c):
    t = time.time()
    if t - _SA[0] < 1.0: return _SA[1]
    rows = c.execute("SELECT a.*,u.name sn FROM star_auc a LEFT JOIN users u ON u
    _SA[1] = [{"id": r["id"], "seller": r["seller"], "sn": str(r["sn"] or "")[:24
               "end": r["end"], "next": sa_next(r)} for r in rows]
    _SA[0] = t; return _SA[1]

@api("/api/sa/list")
def sa_list(b):
    uid, _ = auth(b.get("initData")); c = db()
    if c.execute("SELECT 1 FROM star_auc WHERE status='open' AND end<=? LIMIT 1",
    return {"now": int(time.time()), "list": sa_snapshot(c), "stars": wallet_of(c

@api("/api/sa/create")
def sa_create(b):
    uid, _ = auth(b.get("initData")); c = db(); p = clean_plate(b.get("plate") or
    try: start = int(b.get("start", 0)); mins = int(b.get("mins", 0))
    except (TypeError, ValueError): raise HTTPException(400, "Неверные данные лот
    if tier_py(p) < 4: raise HTTPException(400, "На звёздный аукцион можно выстав
    if mins not in (10, 60, 360, 1440) or not (SA_MIN <= start <= 1000000): raise
    if c.execute("SELECT COUNT(*) FROM star_auc WHERE seller=? AND status='open'"
    if c.execute("UPDATE uniq SET owner=-1 WHERE key=? AND owner=?", (ukey(p), ui
        c.rollback(); raise HTTPException(400, "Этот единственный номер вам не пр
    c.execute("INSERT INTO star_auc(seller,item,start,end) VALUES(?,?,?,?)", (uid
    return {"ok": 1}

@api("/api/sa/bid")
def sa_bid(b):
```

```
    uid, _ = auth(b.get("initData")); c = db(); sa_settle(c); now = int(time.time
    try: aid = int(b["id"]); exp = int(b.get("exp", -1))
    except (KeyError, TypeError, ValueError): raise HTTPException(400, "Неверные
    a = c.execute("SELECT * FROM star_auc WHERE id=? AND status='open' AND end>?"
    if not a: raise HTTPException(400, "Торги уже закончились")
    if a["seller"] == uid: raise HTTPException(400, "Нельзя ставить на свой лот")
    if a["topuid"] == uid: raise HTTPException(400, "Вы уже лидер")
    price = sa_next(a)
    if price != exp: raise HTTPException(400, "Ставка изменилась, попробуйте ещё
    wspend(c, uid, price, "sa-bid:%d" % aid)
    if c.execute("UPDATE star_auc SET top=?,topuid=?,end=MAX(end,?) WHERE id=? AN
                 (price, uid, now + 30, aid, now, a["top"], a["topuid"] or 0)).ro
        c.rollback(); raise HTTPException(400, "Вашу ставку перебили, попробуйте
    if a["topuid"]: wadd(c, a["topuid"], a["top"], "sa-refund:%d" % aid)
    c.commit(); _SA[0] = 0
    if a["topuid"]: notify(a["topuid"], "🔨 Вашу ставку на аукционе за ⭐ перебил
    return {"ok": 1, "price": price, "stars": wallet_of(c, uid)}

@api("/api/sa/cancel")
def sa_cancel(b):
    uid, _ = auth(b.get("initData")); c = db()
    try: aid = int(b["id"])
    except (KeyError, TypeError, ValueError): raise HTTPException(400, "Неверные
    a = c.execute("SELECT * FROM star_auc WHERE id=? AND status='open'", (aid,)).
    if not a or a["seller"] != uid: raise HTTPException(400, "Лот недоступен")
    if c.execute("UPDATE star_auc SET status='cancelled' WHERE id=? AND status='o
        raise HTTPException(400, "На лот уже сделали ставку, снять нельзя")
    gift(c, uid, {"t": "plate", "v": json.loads(a["item"])}); c.commit(); _SA[0]
    return {"ok": 1}

# ---------------------------------------------------------------- витрина: игрок
_SHOP = [0, []]
def shop_snapshot(c):   # список витрины: один раз в 2 секунды на всех игроков ср
    t = time.time()
    if t - _SHOP[0] < 2.0: return _SHOP[1]
    rows = c.execute("SELECT s.id,s.seller,s.item,s.price,s.ts,u.name FROM shop s
    _SHOP[1] = [{"id": r["id"], "seller": r["seller"], "sn": str(r["name"] or "")
    _SHOP[0] = t; return _SHOP[1]

@api("/api/shop/list")
def shop_list(b):
    uid, _ = auth(b.get("initData")); return {"list": shop_snapshot(db())}

@api("/api/shop/create")
def shop_create(b):
    uid, _ = auth(b.get("initData")); c = db(); item = b.get("item") or {}
    try: price = int(b.get("price", 0))
```

```
    except (TypeError, ValueError): raise HTTPException(400, "Неверная цена")
    if not isinstance(item, dict) or item.get("t") not in ("plate", "car") or not
        raise HTTPException(400, "Неверные данные: цена от 1 000")
    need_own(c, uid, item["v"])
    if c.execute("SELECT COUNT(*) FROM shop WHERE seller=? AND status='open'", (u
        raise HTTPException(400, "Не больше 20 предметов на витрине")
    c.execute("INSERT INTO shop(seller,item,price,ts) VALUES(?,?,?,?)", (uid, jso
    _SHOP[0] = 0
    return {"ok": 1}

@api("/api/shop/buy")
def shop_buy(b):
    uid, uname = auth(b.get("initData")); c = db()
    try: sid = int(b["id"]); exp = int(b.get("exp", -1))
    except (KeyError, TypeError, ValueError): raise HTTPException(400, "Неверные
    o = c.execute("SELECT * FROM shop WHERE id=? AND status='open'", (sid,)).fetc
    if not o: raise HTTPException(400, "Уже куплено или снято с продажи")
    if o["seller"] == uid: raise HTTPException(400, "Это ваш предмет")
    if o["price"] != exp: raise HTTPException(400, "Цена изменилась, обновите спи
    if c.execute("UPDATE shop SET status='sold',buyer=? WHERE id=? AND status='op
        raise HTTPException(400, "Уже куплено или снято с продажи")
    item = json.loads(o["item"]); got = int(o["price"] * 0.95)
    gift(c, uid, item); gift(c, o["seller"], {"t": "bal", "v": got}); c.commit();
    notify(o["seller"], "💰 У вас купили!\n%s\nЦена: %s сум. Вам начислено %s сум
    return {"ok": 1}

@api("/api/shop/cancel")
def shop_cancel(b):
    uid, _ = auth(b.get("initData")); c = db()
    try: sid = int(b["id"])
    except (KeyError, TypeError, ValueError): raise HTTPException(400, "Неверные
    o = c.execute("SELECT * FROM shop WHERE id=? AND status='open'", (sid,)).fetc
    if not o or o["seller"] != uid: raise HTTPException(400, "Предмет недоступен"
    if c.execute("UPDATE shop SET status='cancelled' WHERE id=? AND status='open'
        raise HTTPException(400, "Предмет уже куплен")
    gift(c, uid, json.loads(o["item"])); c.commit(); _SHOP[0] = 0
    return {"ok": 1}

# ---------------------------------------------------------------- дуэли между иг
DUEL_STAKES = (100000, 500000, 2000000, 10000000)
DUEL_W = [1, 1.6, 2.6, 4.2, 6.5, 9]   # «сила» класса номера: одинаковые классы =
DUEL_TTL = 86400                       # вызов висит сутки, потом ставка возвращ
_RND = random.SystemRandom()
_DU = [0, []]

def duel_p(ta, tb):
    wa, wb = DUEL_W[min(5, max(0, ta))], DUEL_W[min(5, max(0, tb))]
```

```
    return wa / (wa + wb)

def duel_snapshot(c):
    t = time.time()
    if t - _DU[0] < 2.0: return _DU[1]
    rows = c.execute("SELECT d.id,d.a,d.aplate,d.stake,d.ts,u.name FROM duels d L
    _DU[1] = [{"id": r["id"], "a": r["a"], "an": str(r["name"] or "")[:24], "plat
    _DU[0] = t; return _DU[1]

@api("/api/duel/list")
def duel_list(b):
    uid, _ = auth(b.get("initData")); c = db()
    mine = []
    for r in c.execute("SELECT d.*,ua.name an,ub.name bn FROM duels d LEFT JOIN u
                       "WHERE d.status='done' AND (d.a=? OR d.b=?) ORDER BY d.id
        me_a = r["a"] == uid
        mine.append({"id": r["id"], "win": r["winner"] == uid, "stake": r["stake"
                     "opp": json.loads(r["bplate"] if me_a else r["aplate"]), "on
    return {"list": duel_snapshot(c), "mine": mine, "stakes": list(DUEL_STAKES)}

@api("/api/duel/create")
def duel_create(b):
    uid, _ = auth(b.get("initData")); c = db(); plate = clean_plate(b.get("plate"
    try: stake = int(b.get("stake", 0))
    except (TypeError, ValueError): raise HTTPException(400, "Неверная ставка")
    if stake not in DUEL_STAKES: raise HTTPException(400, "Неверная ставка")
    need_own(c, uid, plate)
    if uid not in HIDDEN and lim_hit("duel", uid, 1, DUEL_DAY): raise HTTPExcepti
    if c.execute("SELECT COUNT(*) FROM duels WHERE a=? AND status='open'", (uid,)
        raise HTTPException(400, "Не больше 3 открытых вызовов")
    c.execute("INSERT INTO duels(a,aplate,stake,ts) VALUES(?,?,?,?)", (uid, json.
    return {"ok": 1}

@api("/api/duel/accept")
def duel_accept(b):
    uid, uname = auth(b.get("initData")); c = db(); bplate = clean_plate(b.get("p
    try: did = int(b["id"])
    except (KeyError, TypeError, ValueError): raise HTTPException(400, "Неверные
    need_own(c, uid, bplate)
    d = c.execute("SELECT * FROM duels WHERE id=? AND status='open'", (did,)).fet
    if not d: raise HTTPException(400, "Вызов уже принят или отменён")
    if d["a"] == uid: raise HTTPException(400, "Нельзя принять свой вызов")
    if uid not in HIDDEN and lim_hit("duel", uid, 1, DUEL_DAY): raise HTTPExcepti
    if d["stake"] >= 2000000 and c.execute("SELECT 1 FROM duels WHERE status='don
        raise HTTPException(400, "Слишком много дуэлей с одним и тем же игроком")
    if c.execute("UPDATE duels SET status='done',b=?,bplate=? WHERE id=? AND stat
        raise HTTPException(400, "Вызов уже принят или отменён")
```

```
    aplate = json.loads(d["aplate"]); pa = duel_p(tier_py(aplate), tier_py(bplate
    a_wins = _RND.random() < pa; winner = d["a"] if a_wins else uid; payout = int
    c.execute("UPDATE duels SET winner=? WHERE id=?", (winner, did))
    gift(c, winner, {"t": "bal", "v": payout}); c.commit(); _DU[0] = 0
    me = str(uname)[:30] or ("ID %d" % uid)
    notify(d["a"], "⚔️ Ваш вызов на дуэль принят!\nСтавка: %s сум\nВаш номер %s п
        fnum(d["stake"]), plate_txt(aplate), plate_txt(bplate), me,
        ("🏆 Вы победили! Вам начислено %s сум." % fnum(payout)) if a_wins else (
    return {"ok": 1, "win": not a_wins, "payout": payout, "opp": aplate, "chance"

@api("/api/duel/cancel")
def duel_cancel(b):
    uid, _ = auth(b.get("initData")); c = db()
    try: did = int(b["id"])
    except (KeyError, TypeError, ValueError): raise HTTPException(400, "Неверные
    d = c.execute("SELECT * FROM duels WHERE id=? AND status='open'", (did,)).fet
    if not d or d["a"] != uid: raise HTTPException(400, "Вызов недоступен")
    if c.execute("UPDATE duels SET status='cancelled' WHERE id=? AND status='open
        raise HTTPException(400, "Вызов уже принят")
    gift(c, uid, {"t": "bal", "v": d["stake"]}); c.commit(); _DU[0] = 0
    return {"ok": 1}

def duel_expire(c):   # вызовы, которые никто не принял за сутки: ставка возвраща
    notes = []
    for d in c.execute("SELECT * FROM duels WHERE status='open' AND ts<?", (int(t
        if c.execute("UPDATE duels SET status='expired' WHERE id=? AND status='op
        gift(c, d["a"], {"t": "bal", "v": d["stake"]})
        notes.append((d["a"], "⌛ Ваш вызов на дуэль никто не принял за сутки.\nС
    if notes: c.commit(); _DU[0] = 0
    for n in notes: notify(*n)

# ------------------------------------------------------------------ ТЮНИНГ И ГОНКИ
CAR_STATS = {   # л.с., Нм, разгон 0-100 (с), масса (кг)
 "Daewoo Matiz": (52, 69, 17, 800), "Daewoo Damas": (63, 88, 20, 1000), "Chevrole
 "Daewoo Tico": (41, 59, 20, 700), "Chevrolet Lacetti": (109, 145, 10.5, 1200), "
 "Ravon Nexia R3": (105, 134, 11.5, 1050), "Daewoo Nexia": (80, 123, 12.5, 1000),
 "Hyundai Accent": (123, 151, 10.4, 1100), "Chevrolet Cobalt": (106, 141, 11.5, 1
 "BYD Song Plus": (197, 325, 8.5, 1800), "BYD Han": (272, 360, 7.9, 1950), "Chevr
 "Chevrolet Monza": (113, 147, 11, 1200), "Kia K5": (180, 230, 9.0, 1450), "Hyund
 "Chevrolet Equinox": (175, 275, 9.0, 1600), "Chevrolet Trailblazer": (177, 260,
 "Chevrolet Malibu XL": (233, 350, 7.4, 1560), "Volkswagen Passat": (190, 320, 7.
 "Chery Tiggo 8 Pro": (254, 390, 7.8, 1750), "Haval Jolion": (143, 210, 10, 1400)
 "BMW X7 M60i": (530, 750, 4.7, 2500), "Mercedes S-Class": (435, 520, 4.9, 2150),
 "Zeekr 009": (544, 700, 4.5, 2800), "Zeekr 001": (544, 768, 3.8, 2300), "Zeekr 7
 "BMW X7 xDrive40i": (380, 520, 5.8, 2400), "Toyota Land Cruiser Prado": (278, 43
 "BMW M3": (510, 650, 3.5, 1730), "Toyota Land Cruiser 200": (309, 439, 8.5, 2600
 "BMW 5 Series": (340, 450, 5.3, 1850), "Audi A6": (340, 500, 5.1, 1900), "Lexus
```

```
 "Range Rover SE 2020": (400, 550, 5.9, 2400), "Range Rover SV 2025": (530, 750,
 "Lamborghini Urus": (650, 850, 3.6, 2200), "Porsche 911 Turbo S": (650, 800, 2.7
 "Mercedes G63 AMG Black Edition": (585, 850, 4.5, 2560), "BMW X7 Alpina XB7": (6
 "Bentley Bentayga": (542, 770, 4.5, 2400), "Aston Martin DB12": (671, 800, 3.6,
 "Mercedes-Maybach GLS 600": (557, 730, 4.9, 2700), "Lamborghini Huracán": (640,
 "Pagani Huayra": (730, 1000, 2.8, 1350), "Koenigsegg Jesko": (1600, 1500, 2.5, 1
 "Bugatti Chiron": (1500, 1600, 2.4, 1995), "Mercedes G63 «Аюб»": (585, 850, 4.5,
 "Mercedes-AMG GT 63 S": (639, 900, 3.2, 2045), "Audi RS7": (600, 800, 3.6, 2065)
 "Lamborghini Aventador SVJ": (770, 720, 2.8, 1525), "Bugatti Veyron": (1001, 125
 "Lamborghini Revuelto": (1015, 807, 2.5, 1772)}
CAR_DEF = [(90, 120, 13, 1100), (150, 200, 9, 1400), (250, 350, 6.5, 1700), (450,
TUNE_PARTS = ("s", "su", "ti", "li", "ae", "ni")   # Stage, подвеска, шины, облег
TUNE_LV = tuple(int(x) for x in os.environ.get("TUNE_LV", "20,35,50").split(","))
TUNE_CH = {1: 1.0, 2: .85, 3: .70, 4: 1.0}              # шанс успешно поднять ча
TUNE_MIN_T = 3                                      # тюнинг только Легендарные и С
TUNE_INS = int(os.environ.get("TUNE_INS", 29))   # страховка за ⭐: при провале у
TUNE_GOLD = {"s": int(os.environ.get("GOLD_STAGE", 77)), "su": 29, "ti": 29, "li"
S_HP = (0, .10, .22, .35, .55); S_NM = (0, .08, .18, .30, .46); L_KG = (0, .03, .

def tn_of(tn):
    try: v = [max(0, min(4, int(x))) for x in tn][:6]
    except Exception: v = []
    return (v + [0] * 6)[:6]

G = 9.81
def _sim(eff, kg, mu, cal, ae=0, ni=0, dist=402.0, to100=False):
    """Физика заезда: мощность, масса, сцепление шин, сопротивление воздуха, пере
    m = kg + 85.0; P = eff * 745.7 * .88 * cal; cda = .62 * (1 - .10 * ae); crr =
    at = mu * G * (1 + .03 * ae); th = (17.0, 30.0, 44.0, 60.0, 78.0); sh = max(.
    v = x = t = 0.0; dt = .01; gear = 0; blk = -1.0; t100 = None; n0 = None
    while t < 90.0:
        if x >= .35 * dist and n0 is None: n0 = t
        boost = 1 + nb if (n0 is not None and t - n0 < 3.0) else 1.0
        F = 0.0 if t < blk else min(m * at, P * boost / max(v, 3.0))
        a = (F - (.5 * 1.2 * cda * v * v + crr)) / m
        v = max(0.0, v + a * dt); x += v * dt; t += dt
        if gear < 5 and v >= th[gear]: gear += 1; blk = t + sh
        if t100 is None and v >= 27.78: t100 = t
        if to100 and t100 is not None: return t100, None
        if x >= dist: return (t100 if t100 is not None else t), t
    return (t100 if t100 is not None else 90.0), 90.0

_CAL = {}
def _calib(hp0, nm0, kg0, t0):   # подгонка под паспортный разгон 0-100 серийной
    key = (hp0, nm0, kg0, t0)
    if key in _CAL: return _CAL[key]
    eff = .65 * hp0 + .35 * nm0 * .72; mu = 1.0
```

```
    while True:
        if _sim(eff, kg0, mu, 6.0, to100=True)[0] > t0 and mu < 2.4: mu *= 1.06;
        lo, hi = .15, 6.0
        for _ in range(22):
            mid = (lo + hi) / 2
            if _sim(eff, kg0, mu, mid, to100=True)[0] > t0: lo = mid
            else: hi = mid
        break
    _CAL[key] = (mu, (lo + hi) / 2); return _CAL[key]

def car_stats(name, t, tn):
    base = CAR_STATS.get(name)
    if not base:
        for k, v in CAR_STATS.items():
            if name.startswith(k.split(" «")[0]) and "«" in name: base = v; break
    if not base: base = CAR_DEF[max(0, min(4, int(t or 0)))]
    hp0, nm0, t0, kg0 = base; s, su, ti, li, ae, ni = tn_of(tn)
    hp = hp0 * (1 + S_HP[s]); nm = nm0 * (1 + S_NM[s]); kg = kg0 * (1 - L_KG[li])
    mu0, cal = _calib(hp0, nm0, kg0, t0); mu = mu0 * (1 + TI_MU[ti]); eff = .65 *
    t100, tt = _sim(eff, kg, mu, cal, ae, ni)
    return {"hp": round(hp), "nm": round(nm), "t100": round(t100, 1), "kg": round

def race_time(st, rnd):
    sg = .025 * (1 - SU_SG[st["tn"][1]])
    return st["tt"] * (1 + max(-3 * sg, min(3 * sg, rnd.gauss(0, sg))))

def race_chance(sa, sb, n=1500):
    r = random.Random(); w = 0
    for _ in range(n):
        if race_time(sa, r) < race_time(sb, r): w += 1
    return w / n

def _find_car(st, car):   # машина игрока в сохранении: по id, иначе первая подхо
    L = st.get("cars") or []
    if car.get("id"):
        for x in L:
            if isinstance(x, dict) and x.get("id") == car["id"]: return x
        return None
    for x in L:
        if isinstance(x, dict) and not x.get("id") and x.get("n") == car.get("n")
           and json.dumps(x.get("pl"), sort_keys=True) == json.dumps(car.get("pl"
    return None

def _ustate(c, uid):
    r = c.execute("SELECT state FROM users WHERE id=?", (uid,)).fetchone()
    try: return json.loads(r["state"]) if r and r["state"] else {}
    except Exception: return {}
```

```
def _usave(c, uid, st): c.execute("UPDATE users SET state=? WHERE id=?", (json.du

def tune_row(c, uid, cid):
    r = c.execute("SELECT s,su,ti,li,ae,ni FROM car_tune WHERE uid=? AND cid=?",
    return [r["s"], r["su"], r["ti"], r["li"], r["ae"] or 0, r["ni"] or 0] if r e

def _tune_apply(c, uid, st):   # сервер - хозяин уровней тюнинга: что бы ни присл
    for x in (st.get("cars") or []):
        if isinstance(x, dict):
            if x.get("id"): x["tn"] = tune_row(c, uid, str(x["id"]))
            else: x.pop("tn", None)

def race_car(c, uid, st, car):
    """Проверка машины для гонки: она есть в гараже игрока; возвращает (машина из
    if not isinstance(car, dict): raise HTTPException(400, "Выберите машину")
    cc = {"n": str(car.get("n") or "")[:60], "t": int(car.get("t") or 0), "m": ca
    x = _find_car(st, cc)
    if not x: raise HTTPException(400, "Этой машины нет в вашем гараже (обновите
    tn = tune_row(c, uid, x["id"]) if x.get("id") else [0] * 6
    return x, car_stats(x.get("n", ""), x.get("t"), tn)

@api("/api/tune")
def tune(b):
    uid, _ = auth(b.get("initData")); c = db(); car = b.get("car") or {}; part =
    if part not in TUNE_PARTS: raise HTTPException(400, "Неверная деталь")
    st = _ustate(c, uid); cc = {"n": str(car.get("n") or ""), "t": int(car.get("t
    x = _find_car(st, cc)
    if not x: raise HTTPException(400, "Этой машины нет в вашем гараже (обновите
    if int(x.get("t") or 0) < TUNE_MIN_T: raise HTTPException(400, "Тюнинг доступ
    if not x.get("id"): x["id"] = "%010x" % random.SystemRandom().getrandbits(40)
    cid = x["id"]; row = tune_row(c, uid, cid); i = TUNE_PARTS.index(part); lvl =
    if lvl >= 4: raise HTTPException(400, "Деталь уже золотая (максимум)")
    gold = lvl == 3
    if not gold and part == "s" and int(st.get("lv") or 1) < TUNE_LV[lvl]: raise
    if gold: ins = False; wspend(c, uid, TUNE_GOLD[part], "tune-gold")
    elif ins: wspend(c, uid, TUNE_INS, "tune-ins")
    ok = True if gold else random.SystemRandom().random() < TUNE_CH[lvl + 1]
    if ok: row[i] = lvl + 1
    elif not ins: row[i] = max(0, lvl - 1)
    c.execute("INSERT OR REPLACE INTO car_tune(uid,cid,s,su,ti,li,ae,ni) VALUES(?
    x["tn"] = list(row); _usave(c, uid, st)
    _pend_add(c, uid, {"id": cid, "n": cc["n"], "t": cc["t"], "m": cc["m"], "pl":
    c.commit()
    return {"ok": 1, "id": cid, "tn": row, "success": ok, "ins": ins, "lost": (no

def _count_pl(st, p): return sum(1 for x in (st.get("col") or []) if _same_pl(x,
```

```
def _pend_add(c, uid, pk, win, newm=0, cnt=0):
    c.execute("INSERT INTO upg_pend(uid,pk,win,newm,ts,cnt) VALUES(?,?,?,?,?,?)",

def _tune_pend(st, d):   # устаревшее сохранение без id/тюнинга машины: вернуть i
    L = [x for x in (st.get("cars") or []) if isinstance(x, dict)]
    for x in L:
        if x.get("id") == d["id"]: x["tn"] = d["tn"]; return
    for x in L:
        if not x.get("id") and x.get("n") == d["n"] and int(x.get("t") or 0) == i
            x["id"] = d["id"]; x["tn"] = d["tn"]; return

def _strip_item(st, item):   # убрать предмет-ставку из сохранения игрока
    v = item.get("v")
    if item.get("t") == "plate":
        col = st.get("col") or []
        for i, x in enumerate(col):
            if _same_pl(x, clean_plate(v)): col.pop(i); return True
    elif item.get("t") == "car":
        L = st.get("cars") or []
        for i, x in enumerate(L):
            if isinstance(x, dict) and ((v.get("id") and x.get("id") == v["id"])
                L.pop(i); return True
    return False

def _item_ok(c, uid, st, item):
    """Предмет-ставка: есть в гараже игрока (и его единственный номер действитель
    if not isinstance(item, dict) or item.get("t") not in ("plate", "car") or not
    if item["t"] == "plate":
        p = clean_plate(item["v"]); need_own(c, uid, p)
        if not any(_same_pl(x, p) for x in (st.get("col") or [])): raise HTTPExce
        return {"t": "plate", "v": p}, tier_py(p)
    v = item["v"]; x = _find_car(st, {"n": v.get("n"), "t": v.get("t"), "m": v.ge
    if not x: raise HTTPException(400, "Этой машины нет в гараже")
    if x.get("pl"): need_own(c, uid, x["pl"])
    return {"t": "car", "v": dict(x)}, int(x.get("t") or 0)

RACE_STAKES = {"bal": (100000, 500000, 2000000, 10000000, 50000000), "stars": (10
RACE_DAY = int(os.environ.get("RACE_DAY", 20))
RACE_TTL = 86400

def _race_cars_view(row_car):
    return {"n": row_car["n"], "t": row_car.get("t"), "pl": row_car.get("pl"), "s

def _race_pay(c, r, winner, loser):   # выплаты победителю: сумы/звёзды/предметы
    typ = r["stype"]
    if typ == "bal":
```

```
        pay = int(r["sval"] * 2 * .95); gift(c, winner, {"t": "bal", "v": pay});
    if typ == "stars":
        pay = int(r["sval"] * 2 * .95); wadd(c, winner, pay, "race-win"); return
    for k in ("aitem", "bitem"):
        if r.get(k): gift(c, winner, json.loads(r[k]))
    return 0

@api("/api/race/create")
def race_create(b):
    uid, _ = auth(b.get("initData")); c = db(); typ = b.get("stype")
    if typ not in ("bal", "stars", "item"): raise HTTPException(400, "Неверная ст
    st = _ustate(c, uid); x, cs = race_car(c, uid, st, b.get("car"))
    if uid not in HIDDEN and lim_hit("race", uid, 1, RACE_DAY): raise HTTPExcepti
    if c.execute("SELECT COUNT(*) FROM races WHERE a=? AND status='open'", (uid,)
    item = None; sval = 0
    if typ == "item":
        item, tr = _item_ok(c, uid, st, b.get("item"))
        if item["t"] == "car" and x is not None and item["v"].get("id") == x.get(
    else:
        try: sval = int(b.get("stake", 0))
        except (TypeError, ValueError): raise HTTPException(400, "Неверная ставка
        if sval not in RACE_STAKES[typ]: raise HTTPException(400, "Неверная ставк
        if typ == "stars": wspend(c, uid, sval, "race-stake")
    cj = {"n": x.get("n"), "t": x.get("t"), "m": x.get("m"), "pl": x.get("pl"), "
    if item:
        _strip_item(st, item); _usave(c, uid, st)
        _pend_add(c, uid, item, -1, 0, _count_pl(st, item["v"]) if item["t"] == "
        if item["t"] == "plate" and tier_py(item["v"]) >= 4: c.execute("UPDATE un
        elif item["t"] == "car" and item["v"].get("pl") and tier_py(clean_plate(i
    c.execute("INSERT INTO races(a,acar,stype,sval,aitem,ts) VALUES(?,?,?,?,?,?)"
    return {"ok": 1, "st": cs}

def _race_view(r, names=None):
    ac = json.loads(r["acar"]); ai = json.loads(r["aitem"]) if r["aitem"] else No
    return {"id": r["id"], "a": r["a"], "an": str((names or {}).get(r["a"], "") o

@api("/api/race/list")
def race_list(b):
    uid, _ = auth(b.get("initData")); c = db()
    rows = c.execute("SELECT r.*,u.name nm FROM races r LEFT JOIN users u ON u.id
    lst = []
    for r in rows:
        v = _race_view(r, {r["a"]: r["nm"]}); lst.append(v)
    mine = []
    for r in c.execute("SELECT r.*,ua.name an,ub.name bn FROM races r LEFT JOIN u
        me_a = r["a"] == uid; res = json.loads(r["result"])
        mine.append({"id": r["id"], "win": r["winner"] == uid, "stype": r["stype"
```

```
    return {"list": lst, "mine": mine, "stakes": {k: list(v) for k, v in RACE_STA

@api("/api/race/odds")
def race_odds(b):
    uid, _ = auth(b.get("initData")); c = db(); st = _ustate(c, uid)
    try: rid = int(b["id"])
    except (KeyError, TypeError, ValueError): raise HTTPException(400, "Неверные
    r = c.execute("SELECT * FROM races WHERE id=? AND status='open'", (rid,)).fet
    if not r: raise HTTPException(400, "Заезд уже недоступен")
    x, cs = race_car(c, uid, st, b.get("car")); ac = json.loads(r["acar"])
    return {"ok": 1, "chance": round(race_chance(cs, ac["st"]) * 100), "st": cs}

@api("/api/race/accept")
def race_accept(b):
    uid, uname = auth(b.get("initData")); c = db()
    try: rid = int(b["id"])
    except (KeyError, TypeError, ValueError): raise HTTPException(400, "Неверные
    r = c.execute("SELECT * FROM races WHERE id=? AND status='open'", (rid,)).fet
    if not r: raise HTTPException(400, "Заезд уже принят или отменён")
    if r["a"] == uid: raise HTTPException(400, "Нельзя принять свой заезд")
    if uid not in HIDDEN and lim_hit("race", uid, 1, RACE_DAY): raise HTTPExcepti
    st = _ustate(c, uid); xb, sb = race_car(c, uid, st, b.get("car"))
    ac = json.loads(r["acar"]); sta = _ustate(c, r["a"])
    xa = _find_car(sta, {"n": ac["n"], "t": ac["t"], "m": ac["m"], "pl": ac["pl"]
    if not xa:   # соперник продал или потерял машину - заезд отменяется, ставка
        _race_refund(c, r, "машина соперника больше не в гараже"); c.commit(); ra
    sa = car_stats(xa.get("n", ""), xa.get("t"), tune_row(c, r["a"], xa["id"]) if
    bitem = None
    if r["stype"] == "item":
        bitem, tb = _item_ok(c, uid, st, b.get("item")); ai = json.loads(r["aitem
        ta_ = tier_py(ai["v"]) if ai["t"] == "plate" else int(ai["v"].get("t") or
        if bitem["t"] != ai["t"] or tb < ta_: raise HTTPException(400, "Нужен пре
        if bitem["t"] == "car" and bitem["v"].get("id") and bitem["v"].get("id")
    elif r["stype"] == "stars": wspend(c, uid, r["sval"], "race-stake")
    if c.execute("UPDATE races SET status='done',b=?,aseen=0 WHERE id=? AND statu
        raise HTTPException(400, "Заезд уже принят или отменён")
    rnd = random.SystemRandom(); tA = race_time(sa, rnd); tB = race_time(sb, rnd)
    chance_a = race_chance(sa, sb)
    a_wins = tA < tB; winner = r["a"] if a_wins else uid; loser = uid if a_wins e
    if bitem:
        _strip_item(st, bitem); _usave(c, uid, st)
        _pend_add(c, uid, bitem, -1, 0, _count_pl(st, bitem["v"]) if bitem["t"] =
    r2 = dict(r); r2["bitem"] = json.dumps(bitem) if bitem else None
    pay = _race_pay(c, r2, winner, loser)
    res = {"a": {"n": ac["n"], "pl": ac["pl"], "st": sa, "t": round(tA, 2), "name
           "pay": pay, "chance_a": round(chance_a * 100), "aitem": json.loads(r["
    an = c.execute("SELECT name FROM users WHERE id=?", (r["a"],)).fetchone(); re
```

```
    c.execute("UPDATE races SET bcar=?,bitem=?,result=?,winner=? WHERE id=?", (js
    rstat_add(c, winner, True); rstat_add(c, loser, False); c.commit()
    stk = ("%s сум" % fnum(r["sval"])) if r["stype"] == "bal" else (("%d ⭐" % r[
    notify(r["a"], "🏁 Ваш заезд принят!\n%s против %s (%s)\nСтавка: %s\n%s" % (a
        "🏆 Вы победили! Награда уже у вас." if a_wins else "💥 Вы проиграли."),
    return {"ok": 1, "win": not a_wins, "res": res, "me_a": False}

def _race_refund(c, r, why=""):   # вернуть ставку создателю заезда
    c.execute("UPDATE races SET status='cancelled' WHERE id=? AND status='open'",
    if r["stype"] == "bal": gift(c, r["a"], {"t": "bal", "v": r["sval"]})
    elif r["stype"] == "stars": wadd(c, r["a"], r["sval"], "race-refund")
    elif r["aitem"]: gift(c, r["a"], json.loads(r["aitem"]))

COSM = {"chrome": 40, "gold": 60, "matte": 25, "pearl": 35, "candy": 45,      # п
        "fl_blue": 30, "fl_green": 30, "fl_pink": 30, "fl_white": 40, "fl_gold":

@api("/api/cosm/list")
def cosm_list(b):
    uid, _ = auth(b.get("initData")); c = db()
    return {"own": [x["k"] for x in c.execute("SELECT k FROM cosm WHERE uid=?", (

@api("/api/cosm/buy")
def cosm_buy(b):
    uid, _ = auth(b.get("initData")); c = db(); k = str(b.get("k") or "")
    if k not in COSM: raise HTTPException(400, "Нет такого товара")
    if c.execute("SELECT 1 FROM cosm WHERE uid=? AND k=?", (uid, k)).fetchone():
    wspend(c, uid, COSM[k], "cosm-" + k)
    c.execute("INSERT OR IGNORE INTO cosm VALUES(?,?,?)", (uid, k, int(time.time(
    return {"ok": 1, "stars": stars_of(c, uid)}

BPC = {"bp_neon": (1, 15), "bp_fire": (1, 35), "bp_ice": (2, 10), "bp_royal": (2,

@api("/api/bp/cosm")
def bp_cosm(b):
    uid, _ = auth(b.get("initData")); c = db(); k = str(b.get("k") or "")
    if k not in BPC: raise HTTPException(400, "Нет такой награды")
    need, lvl = BPC[k]
    if bp_tier(c, uid) < need: raise HTTPException(400, "Нужен Battle Pass " + ("
    try: st = _ustate(c, uid) or {}
    except Exception: st = {}
    if int(st.get("bpx") or 0) // 60 < lvl: raise HTTPException(400, "Уровень ещё
    c.execute("INSERT OR IGNORE INTO cosm VALUES(?,?,?)", (uid, k, int(time.time(
    return {"ok": 1}

@api("/api/race/pending")
def race_pending(b):   # принятый вызов, который создатель ещё не видел (уведомле
    uid, _ = auth(b.get("initData")); c = db()
```

```
    r = c.execute("SELECT r.*,ub.name bn FROM races r LEFT JOIN users ub ON ub.id
    if not r or not r["result"]: return {"p": None}
    return {"p": {"id": r["id"], "win": r["winner"] == uid, "stype": r["stype"],

@api("/api/race/seen")
def race_seen(b):
    uid, _ = auth(b.get("initData")); c = db()
    try: rid = int(b["id"])
    except (KeyError, TypeError, ValueError): raise HTTPException(400, "Неверные
    c.execute("UPDATE races SET aseen=1 WHERE id=? AND a=?", (rid, uid)); c.commi
    return {"ok": 1}

@api("/api/race/cancel")
def race_cancel(b):
    uid, _ = auth(b.get("initData")); c = db()
    try: rid = int(b["id"])
    except (KeyError, TypeError, ValueError): raise HTTPException(400, "Неверные
    r = c.execute("SELECT * FROM races WHERE id=? AND status='open'", (rid,)).fet
    if not r or r["a"] != uid: raise HTTPException(400, "Заезд недоступен")
    _race_refund(c, r); c.commit()
    return {"ok": 1}

def race_expire(c):
    notes = []
    for r in c.execute("SELECT * FROM races WHERE status='open' AND ts<?", (int(t
        _race_refund(c, r); notes.append((r["a"], "⌛ Ваш заезд никто не принял з
    if notes: c.commit()
    for n in notes: notify(*n)

# ---- гонки с ботом: соперник подбирается под вашу машину, выигрыш считается по
BOT_STAKES = {"bal": (100000, 500000, 2000000, 10000000), "stars": (10, 25, 50)}
BOT_DAY = int(os.environ.get("BOT_DAY", 40))                    # заездов с ботом
BOT_BAL_DAY = int(os.environ.get("BOT_BAL_DAY", 200 * 10 ** 6))  # сумма ставок с
BOT_NAMES = ["Жасур", "Азиз", "Тимур", "Бобур", "Санжар", "Рустам", "Дилшод", "Фа
_BOT = {}   # uid -> предложение бота (живёт 10 минут, используется один раз)

def _bot_pick(mine):
    """Выбрать машину бота так, чтобы шанс игрока был 30-70%, и посчитать множите
    rnd = random.SystemRandom(); names = list(CAR_STATS.keys()); rnd.shuffle(name
    for nm in names:
        if "«" in nm or nm in ("Zeekr 9X",): continue
        bst = car_stats(nm, 0, [0] * 6)
        # у бота случайный лёгкий тюнинг: чтобы соперники не были одинаковыми
        bst = car_stats(nm, 0, [rnd.randint(0, 1), 0, 0, 0, 0, 0] if rnd.random()
        ch = race_chance(mine, bst, 1200)
        if best is None or abs(ch - .5) < abs(best[1] - .5): best = (nm, ch, bst)
        if .32 <= ch <= .68: best = (nm, ch, bst); break
```

```
    nm, ch, bst = best
    ch = race_chance(mine, bst, 4000)
    ch = max(.05, min(.95, ch)); mult = round(max(1.01, min(3.5, .95 / (math.ceil
    return nm, ch, bst, mult

@api("/api/race/bot/offer")
def race_bot_offer(b):
    uid, _ = auth(b.get("initData")); c = db(); st = _ustate(c, uid)
    x, cs = race_car(c, uid, st, b.get("car"))
    if uid not in HIDDEN and lim_hit("botoffer", uid, 1, 300): raise HTTPExceptio
    nm, ch, bst, mult = _bot_pick(cs)
    off = {"car": {"n": x.get("n"), "t": x.get("t"), "m": x.get("m"), "id": x.get
    _BOT[uid] = off
    if len(_BOT) > 20000: _BOT.clear()
    return {"ok": 1, "bot": off["bot"], "chance": round(ch * 100), "mult": mult,

@api("/api/race/bot/go")
def race_bot_go(b):
    uid, uname = auth(b.get("initData")); c = db(); st = _ustate(c, uid); typ = b
    off = _BOT.get(uid)
    if not off or time.time() - off["ts"] > 600: raise HTTPException(400, "Соперн
    if typ not in ("bal", "stars"): raise HTTPException(400, "Неверная ставка")
    try: stake = int(b.get("stake", 0))
    except (TypeError, ValueError): raise HTTPException(400, "Неверная ставка")
    if stake not in BOT_STAKES[typ]: raise HTTPException(400, "Неверная ставка")
    x, cs = race_car(c, uid, st, b.get("car"))
    oc = off["car"]
    if oc["n"] != x.get("n") or (oc.get("id") or None) != (x.get("id") or None):
    if uid not in HIDDEN:
        if lim_hit("bot", uid, 1, BOT_DAY): raise HTTPException(400, "Лимит заезд
        if typ == "bal" and lim_hit("botbal", uid, stake, BOT_BAL_DAY): raise HTT
    _BOT.pop(uid, None)
    if typ == "stars": wspend(c, uid, stake, "race-bot")
    rnd = random.SystemRandom(); tA = race_time(cs, rnd); tB = race_time(off["bot
    if win:
        pay = int(stake * off["mult"])
        if typ == "bal": gift(c, uid, {"t": "bal", "v": pay})
        else: wadd(c, uid, pay, "race-bot-win")
    rstat_add(c, uid, win); c.commit()
    res = {"a": {"n": x.get("n"), "pl": x.get("pl"), "st": cs, "t": round(tA, 2),
           "pay": pay, "stake": stake, "stype": typ, "bot": 1}
    return {"ok": 1, "win": win, "res": res, "stars": stars_of(c, uid)}

# эксклюзивные машины: их нельзя выбить в рулетке, они появляются только как сист
EXC = [("Rolls-Royce Spectre", 3.5), ("Lamborghini Aventador SVJ", 3.2), ("Ferrar
       ("Porsche Carrera GT", 3.8), ("McLaren P1", 4.5), ("Lamborghini Revuelto",
ULTRA = [("Mercedes G63 «Аюб»", {"k": 1, "r": "01", "d": "007", "z": "AUB"}), ("Z
```

```
SYSLOT_EVERY = 6 * 3600   # новый эксклюзивный лот не чаще раза в 6 часов (торги

SYSLOT_TXT = ["🔨 Эксклюзив на аукционе: {e} {n}!\nСтарт {p} сум. Торги идут 10 м
              "🏎️ В игре появилась редкая машина — {n}. Такую нельзя выбить в рул
              "⚡ Срочно! {e} {n} выставлен на аукцион.\nСтарт {p} сум. Кто успее
              "👑 Редкий лот: {n}.\nТолько здесь и только 10 минут. Стартовая ста
              "🔥 Аукцион открыт! {e} {n} ждёт нового хозяина.\nНачало торгов: {p
              "💥 Шанс, который бывает нечасто: {n} на аукционе.\nЗа 10 минут реш
SYSLOT_BTN = ["🔨 Сделать ставку", "🏎️ Забрать машину", "⚡ На аукцион", "👑 Уча

def _push_syslot(c, item, now, start=0):
    try:
        k = random.randrange(len(SYSLOT_TXT))
        txt = SYSLOT_TXT[k].format(e=item["v"]["e"], n=item["v"]["n"], p=fnum(sta
        push_users(c, "last_seen>?", (now - 3 * 86400,), txt, SYSLOT_BTN[random.r
    except Exception: log.exception("push syslot")

def system_lot(c):
    now = int(time.time())
    if c.execute("SELECT 1 FROM auctions WHERE seller=0 AND status='open'").fetch
    r = c.execute("SELECT v FROM meta WHERE k='syslot'").fetchone()
    if r and now - int(r[0]) < SYSLOT_EVERY: return
    n, m = random.choice(EXC)   # Аюб и Бибиша сами не появляются - только по ком
    item = {"t": "car", "v": {"n": n, "e": "🏎️", "t": 3, "m": m, "pl": None}}; st
    c.execute("INSERT INTO auctions(seller,item,start,end) VALUES(0,?,?,?)", (jso
    c.execute("INSERT OR REPLACE INTO meta(k,v) VALUES('syslot',?)", (str(now),))
    _push_syslot(c, item, now, start)

def launch_ultra(c, idx):
    """Команда админа: запустить супер-лот (1 - Аюб, 2 - Бибиша). Возвращает текс
    if not (1 <= idx <= len(ULTRA)): return "Номер лота: 1 - %s, 2 - %s" % (ULTRA
    now = int(time.time()); n, pl = ULTRA[idx - 1]
    if c.execute("SELECT 1 FROM auctions WHERE seller=0 AND status='open'").fetch
    if is_uniq(pl) and c.execute("SELECT 1 FROM uniq WHERE key=?", (ukey(pl),)).f
    item = {"t": "car", "v": {"n": n, "e": "🚙", "t": 4, "m": 2, "pl": pl}}
    c.execute("INSERT INTO auctions(seller,item,start,end) VALUES(0,?,?,?)", (jso
    c.execute("INSERT OR REPLACE INTO meta(k,v) VALUES('syslot',?)", (str(now),))
    _push_syslot(c, item, now, 600_000_000)
    return "✅ Аукцион запущен: %s, старт 600 000 000 сум, 10 минут. Всем игрокам

# ---------------------------------------------------------------- уход за базой:
def cleanup(c):   # раз в час: убрать старые закрытые обмены и торги, не раздувать
    old = int(time.time()) - 14 * 86400; n = 0
    n += c.execute("DELETE FROM offers WHERE id IN (SELECT id FROM offers WHERE s
    n += c.execute("DELETE FROM auctions WHERE id IN (SELECT id FROM auctions WHE
    n += c.execute("DELETE FROM duels WHERE id IN (SELECT id FROM duels WHERE sta
    n += c.execute("DELETE FROM shop WHERE id IN (SELECT id FROM shop WHERE statu
```

```
    c.commit()
    c.execute("PRAGMA wal_checkpoint(PASSIVE)")
    if n: log.info("чистка базы: удалено старых записей: %d", n)

def _backups():
    try: return sorted(f for f in os.listdir(BK_DIR) if f.startswith("game-") and
    except OSError: return []

def backup_db(tag="auto"):
    """Копия базы (безопасная, пока игроки играют) со сжатием в папку backups ряд
    os.makedirs(BK_DIR, exist_ok=True)
    size = os.path.getsize(DB_PATH) if os.path.exists(DB_PATH) else 0
    free = shutil.disk_usage(BK_DIR).free
    if free < size * 2.2 + 50e6: raise RuntimeError("мало места на диске (свободн
    stamp = time.strftime("%Y%m%d-%H%M%S", time.gmtime())
    tmp = os.path.join(BK_DIR, "tmp-%s.db" % stamp); out = os.path.join(BK_DIR, "
    src = sqlite3.connect(DB_PATH, timeout=30); dst = sqlite3.connect(tmp)
    try: src.backup(dst, pages=2000, sleep=0.001)
    finally: dst.close(); src.close()
    try:
        with open(tmp, "rb") as f, gzip.open(out + ".part", "wb", 6) as g: shutil
        os.replace(out + ".part", out)
    finally:
        for x in (tmp, out + ".part"):
            try: os.remove(x)
            except OSError: pass
    for tagged, keep in ((True, BK_KEEP), (False, 5)):   # автокопий храним BK_KE
        fs = [f for f in _backups() if f.endswith("-auto.db.gz") == tagged]
        for f in fs[:-keep]:
            try: os.remove(os.path.join(BK_DIR, f))
            except OSError: pass
    return out

def auto_backup():   # не чаще раза в ~20 часов
    fs = [f for f in _backups() if f.endswith("-auto.db.gz")]
    if fs and time.time() - os.path.getmtime(os.path.join(BK_DIR, fs[-1])) < 20 *
    c = db()
    if not c.execute("SELECT 1 FROM users LIMIT 1").fetchone(): return
    p = backup_db("auto"); log.info("копия базы: %s (%d КБ)", os.path.basename(p)

DB_CHECK = ["ещё не проверялась", 0]
def integrity():
    r = db().execute("PRAGMA quick_check").fetchone()[0]
    DB_CHECK[0] = r; DB_CHECK[1] = time.time()
    if r != "ok": log.error("ПРОВЕРКА БАЗЫ НАШЛА ПРОБЛЕМУ: %s", r)
    return r
```

```
def _mb(n): return "%.1f МБ" % (n / 1048576)

def db_report():   # текст для команды /stat в боте
    c = db(); now = int(time.time()); out = []
    one = lambda q, a=(): c.execute(q, a).fetchone()[0]
    tot = one("SELECT COUNT(*) FROM users")
    out.append("👥 Игроков: %d · за 24 ч: %d · за 7 дней: %d" % (tot, one("SELECT
    sz = os.path.getsize(DB_PATH) if os.path.exists(DB_PATH) else 0; wal = os.pat
    try: free = shutil.disk_usage(os.path.dirname(os.path.abspath(DB_PATH))).free
    except OSError: free = 0
    out.append("💾 База: %s (журнал %s) · свободно на диске: %s" % (_mb(sz), _mb(
    big = c.execute("SELECT id,length(CAST(state AS BLOB)) l FROM users WHERE sta
    avg = one("SELECT IFNULL(AVG(length(CAST(state AS BLOB))),0) FROM users WHERE
    out.append("📦 Сохранение игрока: в среднем %d КБ, самое большое %d КБ (ID %s
    out.append("🎁 Подарков в очереди: %d · 🤝 открытых обменов: %d · 🔨 открыты:
        one("SELECT COUNT(*) FROM gifts"), one("SELECT COUNT(*) FROM offers WHERE
    out.append("🧪 Проверка целостности: %s%s" % (DB_CHECK[0], (" (" + time.strft
    fs = _backups()
    if fs:
        p = os.path.join(BK_DIR, fs[-1]); out.append("🗄 Копий базы: %d · последн
    else: out.append("🗄 Копий базы пока нет (первая создаётся автоматически; сде
    rows = [x for x in list(_LAT) if x[0] > time.time() - 300]
    if rows:
        ms = sorted(x[2] for x in rows); n = len(ms); pc = lambda q: ms[min(n - 1
        out.append("⚡ За 5 минут: запросов %d (%.1f/сек) · ответ в среднем %d мс
        by = {}
        for _, p, m in rows: by.setdefault(p, []).append(m)
        top = sorted(by.items(), key=lambda kv: -len(kv[1]))[:5]
        out.append("   чаще всего: " + ", ".join("%s ×%d (%d мс)" % (p.replace("/
    else: out.append("⚡ За 5 минут запросов не было")
    return "\n".join(out)

def _bg():   # фоновый поток: торги, системные лоты, чистка и копии (не зависит о
    try: bibisha_purge()
    except Exception: log.exception("bibisha_purge")
    try: bal_backfill()
    except Exception: log.exception("bal_backfill")
    try: uniq_backfill()
    except Exception: log.exception("uniq_backfill")
    for f in os.listdir(BK_DIR) if os.path.isdir(BK_DIR) else []:
        if f.startswith("tmp-") or f.endswith(".part"):
            try: os.remove(os.path.join(BK_DIR, f))
            except OSError: pass
    last_clean = last_bk = last_chk = 0
    while True:
        try: c = db(); settle(c); sa_settle(c); duel_expire(c); race_expire(c); s
        except Exception: log.exception("торги")
```

```
        try: weekend_event(c); daily_reminders(c)
        except Exception: log.exception("рассылки")
        try: ton_poll()
        except Exception as e: log.debug("ton_poll: %s", e)
        t = time.time()
        if t - last_clean > 3600:
            last_clean = t
            try: cleanup(db())
            except Exception: log.exception("чистка базы")
        if t - last_bk > 3600:
            last_bk = t
            try: auto_backup()
            except Exception: log.exception("копия базы")
        if t - last_chk > 86400:
            last_chk = t
            try: integrity()
            except Exception: log.exception("проверка базы")
        time.sleep(5)

if not os.environ.get("NO_BG"): threading.Thread(target=_bg, daemon=True, name="b
```